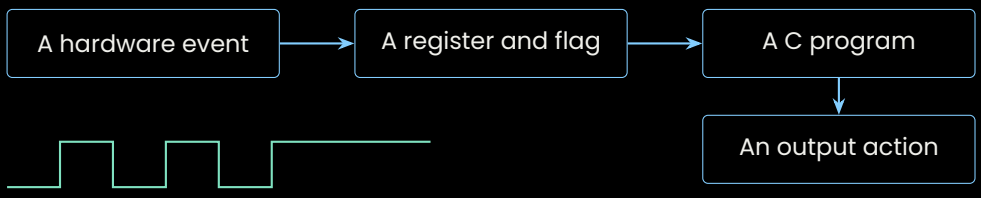
COLLEGE OF ENGINEERING TRIVANDRUM

S5 ELECTRICAL AND COMPUTER ENGINEERING

# Microcontrollers and Embedded Systems

PCEOT501 / Module 3

Serial communication, interrupts, interfacing and motor control




**A guide for understanding and exam preparation**

Complete syllabus coverage, worked numericals, explained Keil C51 programs, vector diagrams, answered practice and a final revision sheet. Both editions contain the same study material.

Pure black edition • Prepared 7 October 2026

Based on the official syllabus and Prof. Murali Krishnan's classroom HTML. Independently checked against architecture documentation, device datasheets and compiler references. This is an independently prepared study aid, not an official CET publication.

## Contents

**Scope, assumptions and how to use this guide** 4

**1 The minimum prerequisite toolkit** 5
1.1 Think in both directions: hardware to C, C to hardware . . . 5
1.2 Ports, SFRs and bit operations . . . 5
1.3 C51 types, memory and ownership . . . 6
1.4 Clock and timer refresher . . . 6

**2 Serial communication and the 8051 UART** 8
2.1 Serial foundations: bytes, frames and agreement . . . 8
2.2 UART versus RS232; connections through MAX232 . . . 9
2.3 SBUF, SCON and PCON . . . 10
2.4 Timer 1 baud generation and baud error . . . 11
2.5 Polling serial programming: complete C51 examples . . . 12
2.6 Receive buffering, lost frames and deadlines . . . 14

**3 Interrupts, priority and reliable Embedded C** 15
3.1 What an interrupt does, and why it is needed . . . 15
3.2 Registers and vector reference . . . 16
3.3 Interrupt entry, return, pending requests and latency . . . 17
3.4 External hardware interrupts and button bounce . . . 17
3.5 Priority, nesting and simultaneous requests . . . 20
3.6 Keil C51 ISR syntax and shared data . . . 21
3.7 The serial interrupt: two flags, one vector . . . 22
3.8 A complete command interface with a receive queue . . . 22
3.9 A nonblocking transmit queue: correcting the advanced exercise . . . 25
3.10 Timing and design questions the exam can ask . . . 26

**4 LCD interfacing** 27
4.1 What the LCD interface controls . . . 27
4.2 Commands, addresses and write timing . . . 28
4.3 Complete eight-bit output program . . . 29
4.4 Four-bit operation and busy-flag reading . . . 30

**5 ADC and DAC interfacing** 30
5.1 Analogue quantities and digital codes . . . 30
5.2 ADC0804: selected device and handshake . . . 31
5.3 DAC0808: current output and voltage conversion . . . 33

**6 Relay and motor interfacing** 35
6.1 Logic control versus load power . . . 35
6.2 Relay: contacts, transistor driver and flyback . . . 35
6.3 Stepper motors and excitation sequences . . . 36
6.4 DC motor direction control using an H bridge . . . 39

**7 Pulse-width modulation using the 8051** 41
7.1 What PWM controls . . . 41
7.2 Classic software PWM versus derivative hardware PWM . . . 41
7.3 Timer 0 mode-2 PWM: a complete program . . . 41

7.4 Timing limits, resolution and alternative calculations . . . . . 43

**8 Answered exam practice and classroom exercises 45**
8.1 Three-mark questions: serial communication . . . . . 45
8.2 Three-mark questions: interrupts and interfaces . . . . . 46
8.3 Nine-mark full questions with developed answers . . . . . 48
8.4 Code reading and debugging: the eight classroom faults . . . . . 51
8.5 Classroom lab traces, viva answers and extensions . . . . . 53
8.6 The classroom graded practice, answered and mapped . . . . . 53

**9 Final revision sheet and completion checklist 54**
9.1 Syllabus completion checklist . . . . . 55

**10 Discrepancy appendix: corrections and simplifications 56**

**11 Resources mapped to the syllabus 59**
11.1 Prescribed textbooks: what was accessible . . . . . 59
11.2 Primary architecture and compiler documentation . . . . . 60
11.3 Device references for each interface . . . . . 61
11.4 Supplementary university study material . . . . . 62

## Scope, assumptions and how to use this guide

The controlling source is the official Electrical and Computer Engineering syllabus: PCEOT501 begins on physical PDF page 128, Module 3 is on page 129, and the ESE pattern is on page 130. Module 3 has **12 contact hours**. The timer/counter syllabus belongs to Module 2; the timer material here is only the prerequisite needed for baud rates, delays and PWM. ARM belongs to Module 4.

**Exam format.** The ESE is 60 marks in 2 hours 30 minutes. Part A requires all eight questions: two per module, each worth 3 marks. Part B gives two full questions per module; answer one, worth 9 marks, with at most three subdivisions. Thus Module 3 contributes 6 marks in Part A and a selected 9-mark full question in Part B. Practice formats here follow that structure; they are neither predictions nor official marking schemes.

**Learning sequence.** Refresh ports and timers; learn framing and RS232 wiring; configure the UART; replace blocking waits with interrupts; reason about concurrency; interface displays and converters; then drive loads and generate PWM. Read each program with its connections before trying to memorise register values. Use the problems after each topic, then attempt the practice bank without looking at its answers.

**Processor, compiler and circuit contract**

Unless a section explicitly changes it: a **classic 12-clock 8051-compatible core**, a **5 V logic supply**, **11.0592 MHz crystal**, internal program memory and **Keil C51** with `reg51.h`. The examples are separate programs: do not link several `main` functions or several handlers for one vector together. GPIO examples assume no external `MOVX` memory bus. The PWM worked program explicitly uses **12 MHz** for exact 250 $\mu$s timer ticks.
The classic 8051 has five interrupt vectors and two priority levels. An 8052 adds Timer 2; modern derivatives can add faster cores, FIFOs, extra UARTs, different baud generators and hardware PWM. Their datasheets override these assumptions. The AT89S51 datasheet is a useful compatible-device check, but its watchdog, dual DPTR and POF bit are derivative additions, not original-8051 features.

**Code status.** Complete example files and their local headers are included in the source bundle. Keil-only keywords are labelled. C51 is unavailable in this environment, so no native C51 compilation or physical-board test is claimed. Static review and independent mathematical/model checks are recorded in `VALIDATION.md`. Timing affected by compiler-generated instructions is explicitly described as nominal or conservative.

### Syllabus coverage map

Page references below are generated from the same LaTeX labels as the contents. Printed page numbers include the unnumbered cover as page 1, so they also match physical PDF page numbers.


| Syllabus topic | Main section | Page |
|---|---|---|
| Basics of serial communication | 2.1 | 8 |
| 8051 connections to RS232 | 2.2 | 9 |
| Serial-port programming in Embedded C | 2.5 | 12 |
| 8051 interrupts | 3 | 15 |
| External hardware interrupts | 3.4 | 17 |
| Serial communication interrupt | 3.7 | 22 |

| Syllabus topic | Main section | Page |
|---|---|---|
| Interrupt priority | 3.5 | 20 |
| Interrupt programming in Embedded C | 3.6 | 21 |
| LCD interfacing | 4 | 27 |
| ADC interfacing | 5.2 | 31 |
| DAC interfacing | 5.3 | 33 |
| Relay interfacing | 6.2 | 35 |
| Stepper-motor interfacing | 6.3 | 36 |
| DC-motor interfacing | 6.4 | 39 |
| PWM using the 8051 | 7 | 41 |

**Classroom continuity.** The supplied HTML by Prof. Murali Krishnan contains 23 slide sections. Its generated content includes 8 labs, 8 debugging exercises, 15 practice problems and 28 quiz questions. Their examples, hardware-to-C reasoning, flag emphasis, priority cases and command-interface pattern are developed here; supplementary interfacing topics come from the syllabus and device references. The discrepancy appendix, page 56, separates corrections from teaching simplifications.

# 1 The minimum prerequisite toolkit

## 1.1 Think in both directions: hardware to C, C to hardware

An interface joins devices that differ in representation, voltage, timing or drive capability. A sensor produces voltage; an ADC produces a binary code; a UART turns a byte into timed bits; a motor needs current through windings. The microcontroller decides *what* should happen, while interface hardware makes the decision electrically meaningful.

Keep the classroom chain: **hardware event** → **peripheral/pin** → **register/flag** → **CPU response** → **C code** → **output**. To debug, trace the chain backwards. If no character appears, ask whether the program wrote SBUF, whether Timer 1 runs, whether TxD changes, whether the converter is powered, and whether the terminal uses the same format.

## 1.2 Ports, SFRs and bit operations

An SFR is a special function register mapped into the 8051's direct-address SFR space. It may represent a latch, a control register, a status register or a hardware action. Reading and writing the same address need not have the same meaning; SBUF is the central example.

**Quasi-bidirectional ports.** On classic P1, P2 and P3, write 1 to a port latch before using its pin as an input. Writing 0 actively pulls it low. The internal pull-up permits input use after writing 1; it does not make the pin a power output. P0 is open-drain in general-purpose GPIO mode and needs external pull-ups for logic-high output. P3 alternate functions require their corresponding port latches to be released (1). Do not assume a modern push-pull port model.

**Read versus read-modify-write.** Instructions that modify a port can read its latch rather than the external pin. This prevents a loaded output from corrupting a neighbouring output, but means a port-modifying expression is not a reliable way to measure a pin. For an ADC, set the data port to `0xFF` and then perform a plain read. For actuator control, maintain a software command value rather than assuming a port read proves the physical load is on.

| C operation | Meaning and reason |
|---|---|
| `SCON = 0x50;` | Write a complete known configuration. Appropriate during initialisation. |
| `PCON \|= 0x80;` | Set bit 7 without replacing all the other bits. |
| `PCON &= 0x7F;` | Clear bit 7; PCON is not bit-addressable. |
| `TMOD = (TMOD & 0x0F) \| 0x20;` | Set Timer 1 mode 2 while preserving Timer 0's low nibble. |
| `sbit LED = P1^0;` | Keil C51 declaration of a named bit. The caret here selects a bit; it is not a run-time XOR operation. |
| `P1 = (unsigned char)~c;` | Complement an 8-bit pattern for LEDs connected from +5 V through resistors to sinking GPIO pins. |

### 1.3 C51 types, memory and ownership

`unsigned char` is 8 bits (0–255), `unsigned int` is 16 bits (0–65535), and `unsigned long` is 32 bits in C51. Use unsigned types for bit patterns. Eight-bit counters wrap after 255; that may be intended, but is not an overflow alarm. A `bit` variable stores a single bit. `sfr`, `sbit`, `code`, `interrupt n` and `using n` are C51 extensions.

The `code` memory qualifier places constant lookup tables/strings in program memory, saving the classic 8051's 128-byte RAM. Use explicit declarations and prototypes. A pointer must be compatible with the memory space it accesses; C51 generic pointers can represent several spaces but cost more than memory-specific pointers. The simple pointer arguments in these examples are ordinary generic pointers; constant tables are accessed directly by index.

**Assign an owner to every peripheral.** Timer 1 belongs to the UART baud clock in serial examples. Timer 0 belongs either to a polling delay *or* to the PWM ISR. A UART flag belongs either to a polling function or to an ISR. Combining individually correct examples without reconciling those owners produces subtle failures.

### 1.4 Clock and timer refresher

For a 12-clock core,

$$f_{\mathrm{MC}} = \frac{f_{\mathrm{osc}}}{12}, \qquad t_{\mathrm{MC}} = \frac{12}{f_{\mathrm{osc}}}.$$

At 11.0592 MHz, $f_{\mathrm{MC}} = 921600$ Hz and $t_{\mathrm{MC}} = 1.085069\ \mu$s. At 12 MHz, one machine cycle is $1\ \mu$s. A timer in timer mode increments once per machine cycle; counter mode counts sampled external transitions and is not used for the delay examples.

In Timer mode 1, THx:TLx forms a 16-bit counter. Starting at reload $R$, it needs $65536 - R$ increments to overflow. Mode 2 uses an 8-bit TLx counter that automatically reloads from THx: $256 -$ THx increments per repeated overflow. `TRx` starts/stops; `TFx` records overflow. `TMOD` is byte-addressable, while `TCON` is bit-addressable.

**Worked example: A conservative millisecond delay**

At 11.0592 MHz, 1 ms corresponds to $921.6$ increments. Choose $922$; then $R = 65536 - 922 = 64614 = \mathrm{FC66_H}$. The hardware count time is

$$922\frac{12}{11059200} = 1.000434028\ \text{ms}.$$

The function below takes *longer* because setup, polling, stopping and looping add instructions. It is suitable for conservative LCD waits and slow demonstrations, not an exact frequency specification.

**Shared header: a polling Timer 0 delay**

```
#ifndef TIMER_DELAY_H
#define TIMER_DELAY_H
#include <reg51.h>
/* Keil C51; 11.0592 MHz, 12 clocks/machine cycle.
   Exclusively owns Timer 0. Blocking delay, with software overhead. */
static void delay_ms(unsigned int ms)
{
    ET0 = 0;
    TMOD = (TMOD & 0xF0) | 0x01;
    while (ms != 0) {
        TR0 = 0;
        TF0 = 0;
        TH0 = 0xFC;
        TL0 = 0x66;         /* 65536 - 922 = 0xFC66. */
        TR0 = 1;
        while (TF0 == 0) { }
        TR0 = 0;
        TF0 = 0;
        --ms;
    }
}
#endif
```

The header is self-contained through `reg51.h`. It preserves Timer 1's configuration, disables the Timer 0 interrupt and clears TF0 explicitly because no ISR vectors to clear it. Calling this function while another subsystem uses Timer 0 would destroy that subsystem's timebase.

**What the exam can ask; common mistakes**

Explain why input ports are written with ones; decode a masked TMOD write; calculate a timer reload; distinguish a C51 extension from ISO C. Do not count oscillator ticks as machine cycles, put a large constant table unnecessarily in RAM, or use a delay loop whose exact time you cannot justify from generated instructions.

Verification and further reading: Intel classic hardware description [INTEL]; AT89S51 pin/port and electrical specifications [S51]; official Keil memory/type references [KEILTYPE], [KEILMEM].

# 2 Serial communication and the 8051 UART

## 2.1 Serial foundations: bytes, frames and agreement

**Serial communication** sends bits successively over a signal line. Parallel communication sends several bits on separate lines in the same transfer interval. Serial saves pins, conductors and connector size; parallel can transfer more bits per clock but must manage timing skew between lines. Neither method is universally faster: compare the complete link, its clock and its overhead.

| Distinction | First form | Second form |
|---|---|---|
| Synchronous / asynchronous | A clock signal or clock recovery defines timing. Classic serial mode 0 provides a shift clock on TxD. | A UART frame resynchronises on a start bit; both ends agree on baud and format. |
| Simplex / half duplex | Simplex: one direction only. | Half duplex: both directions, one at a time. |
| Full duplex | Separate transmit/receive paths allow simultaneous traffic. | Classic 8051 UART modes 1–3 support this; software can still choose a blocking design. |
| Bit rate / baud rate | Bit rate counts bits per second. | Baud counts symbols per second. For binary UART signalling, one symbol carries one bit, so their numerical values agree. |

**Asynchronous 8N1** means 8 data bits, no parity and 1 stop bit. An idle logic-level UART line is high. A low start bit creates a falling edge. Eight data bits follow **least significant bit first**, then a high stop bit. The receiver uses the start edge to align its sampling. The classic mode-1 receiver samples at 16 times the baud rate and makes a majority decision from three samples near each bit centre. A false start is rejected if the start sample is not low.

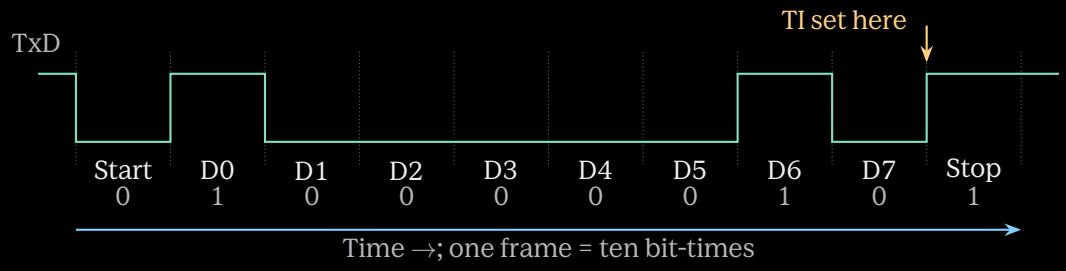



**Figure:** Mode-1 frame for ASCII ‘A’, $41_{\mathrm{H}} = 01000001_2$ when conventionally written MSB first. Data on the wire is $1, 0, 0, 0, 0, 0, 1, 0$. TI is set at the *start* of the stop bit; the stop interval still has to appear on the wire.

**Worked example: Bit time, frame time and useful rate**

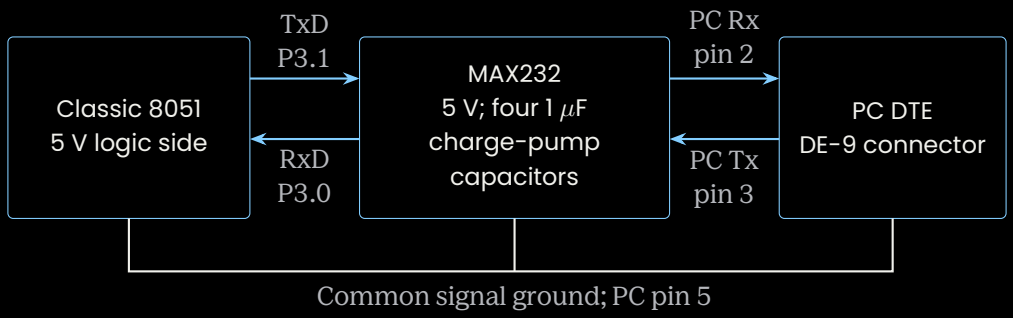
At $B = 9600$ baud, $t_b = 1/B = 104.166667\ \mu$s. An 8N1 frame takes $10/B = 1.041666667$ ms, giving at most $B/10 = 960$ characters/s on a continuously occupied line. Useful data rate is $960 \times 8 = 7680$ bit/s and framing efficiency is $8/10 = 80\%$.
Forty characters occupy $400/9600 = 41.666667$ ms of wire time. The 12-character string `8051 READY\r\n` occupies 12.5 ms. At 4800 baud every time doubles. Inter-character gaps and software delays increase total time; a sender waiting for TI may return before its last stop bit has completely elapsed.

**Parity and framing errors.** Parity adds a check bit according to odd/even convention; it does not provide reliable delivery or error correction. Classic mode 1 has no automatic parity-bit slot beyond its eight data bits. Modes 2/3 offer a ninth bit that software can use for parity or addressing. Both ends must agree on format; a MAX232 does not add parity or repair a bad frame.

## 2.2 UART versus RS232; connections through MAX232

The UART is the on-chip logic that assembles and shifts bytes. RS232 describes an external serial electrical interface and related signals. On the logic side, high means a positive logic voltage. On an RS232 data line, a **mark** (logic 1/idle) is negative and a **space** (logic 0) is positive. For ordinary exam descriptions use valid receiver regions below $-3$ V and above $+3$ V; the range between them is undefined. Common cable amplitudes are approximately $\pm 5$ to $\pm 12$ V, not a compulsory $\pm 12$ V supply.

An **RS232 line driver/receiver** such as the 5 V TI MAX232 translates voltage and polarity in both directions. Its charge pump uses external capacitors to generate its internal positive and negative driver rails. It does not generate baud or understand characters.



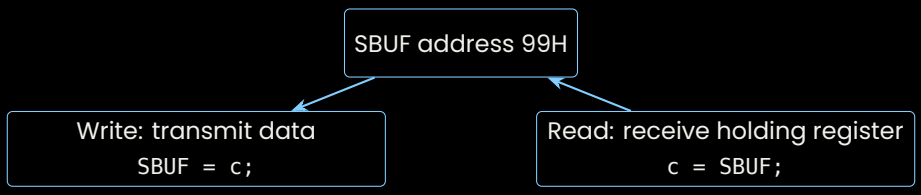

| Connection (MAX232 16-pin package) | Purpose |
|---|---|
| 8051 P3.1 → T1IN (11) | Logic-level transmit input. |
| T1OUT (14) → PC DE-9 pin 2 | PC DTE receives the converted RS232 data. |
| PC DE-9 pin 3 → R1IN (13) | PC DTE transmit output enters the receiver. |
| R1OUT (12) → 8051 P3.0 | Translated logic-level receive signal. |
| VCC (16), GND (15) | +5 V and common ground; supply bypass capacitor nearby. |
| C1+ (1), C1− (3); C2+ (4), C2− (5) | Two flying capacitors; reservoir capacitors at VS+ (2) and VS− (6), wired as the chosen datasheet application shows. |

**Connector convention.** These pin numbers refer to a *PC DTE* DE-9 port, commonly called DB9. A board connector and cable must be wired to preserve the end-to-end path above. DTE/DCE conventions and null-modem cables matter: “always cross pins 2 and 3” is not a universal cable rule. Count pins from the correct connector view; a solder-side drawing reverses the apparent order. A three-wire connection assumes terminal hardware flow control is disabled.

**Adapter convention.** A USB-to-RS232 adapter has RS232 line levels and needs the MAX232. A USB-to-TTL UART adapter already has logic-level TX/RX and normally bypasses the MAX232, with compatible voltage and a common ground. Verify the adapter’s specification. The original MAX232 uses 1 μF capacitors; MAX232A/MAX3232 variants have different capacitor and supply requirements.

### 2.3 SBUF, SCON and PCON

**SBUF (99H)** represents two distinct byte registers. A write loads the transmit path and requests transmission; a read accesses the received-byte holding register. Writing 'X' then reading SBUF does not read 'X' back. Neither writing SBUF nor reading SBUF automatically clears TI/RI. Reception has a shift register plus a holding register, but no multi-byte FIFO; read SBUF only when RI indicates a new accepted byte.




**SCON (98H), bit-addressable, bits 7 to 0:**

| SM0 bit 7 | SM1 bit 6 | SM2 bit 5 | REN bit 4 | TB8 bit 3 | RB8 bit 2 | TI bit 1 | RI bit 0 |
|---|---|---|---|---|---|---|---|

| Bit | Meaning on the classic core |
|---|---|
| SM0, SM1 | Select serial mode 0, 1, 2 or 3. |
| SM2 | In modes 2/3, multiprocessor filtering: RI is suppressed for a received ninth bit of 0. In mode 1, RI is suppressed if SM2=1 and the stop bit is invalid (0). Use SM2=0 for ordinary 8N1. In mode 0, keep SM2=0. |
| REN | Receiver enable, set/cleared by software. REN=0 prevents reception. |
| TB8 | Ninth transmitted data bit in modes 2/3. Unused for the mode-1 data frame. |
| RB8 | Ninth received bit in modes 2/3; received stop-bit value in mode 1 with SM2=0. Not used in mode 0. |
| TI | Set by hardware at the end of the eighth bit in mode 0, and **at the beginning of the stop bit** in modes 1–3. Software clears it. It does not indicate an acknowledgement. |
| RI | Set when a received frame is accepted (mode 1 near the middle of the stop-bit interval, not after a full stop interval). Software clears it after reading the holding register. |

| SM0 SM1 | Mode | Format / pins | Baud |
|---|---|---|---|
| 0 0 | 0 | 8-bit synchronous shift register. RxD is data, TxD is clock. | $f_{\text{osc}}/12$ |
| 0 1 | 1 | Start + 8 data + stop; TxD out, RxD in. | Variable (Timer 1 here) |
| 1 0 | 2 | Start + 9 data + stop. | $2^{\text{SMOD}} f_{\text{osc}}/64$ |
| 1 1 | 3 | Same frame as mode 2. | Variable (Timer 1 here) |

**SCON=50H** is $01010000_2$: SM0=0, SM1=1, SM2=0, REN=1, and TB8/RB8/TI/RI initially clear. `SCON=40H` selects the same mode with REN=0; transmission works but reception does not. A later full SCON assignment also clears any pending flags, so do it during initialisation, not casually during live traffic.

**PCON (87H), not bit-addressable.** For the classic CMOS 80C51 layout: bit 7 SMOD; bits 6–4 reserved; bit 3 GF1; bit 2 GF0; bit 1 PD (power down); bit 0 IDL (idle). The original HMOS 8051 lacks the CMOS low-power interpretation. SMOD doubles UART baud in modes 1–3. Use `PCON |= 0x80;` to set it or `PCON &= 0x7F;` to clear it. Do not write `sbit SMOD = PCON^7;` as if PCON were bit-addressable. On AT89S51, PCON.4 is POF, an explicitly device-specific addition.

### 2.4 Timer 1 baud generation and baud error

In timer mode on a 12-clock core, Timer 1 increments at $f_{\text{osc}}/12$. In mode 2 the interval between repeated overflows is $N = 256 - \text{TH1}$ counts. UART modes 1/3 divide the overflow rate by 32 when SMOD=0 or 16 when SMOD=1:

$$B = \frac{2^{\text{SMOD}}}{32}\,\frac{f_{\text{osc}}}{12(256-\text{TH1})} = \frac{2^{\text{SMOD}} f_{\text{osc}}}{384N}, \qquad N^{*} = \frac{2^{\text{SMOD}} f_{\text{osc}}}{384 B_{\text{target}}}.$$

TH1 must be an integer 0–255, so $N$ must be an integer 1–256. If $N^{*}$ is not integral, calculate the baud for the feasible floor and ceiling counts and choose the smaller absolute baud error, rather than just rounding a displayed reload. If no feasible count gives an acceptable error, change the clock/generator or target rate.

$$\text{signed baud error} = 100\frac{B_{\text{actual}} - B_{\text{target}}}{B_{\text{target}}}\ \%.$$

The oscillator's real tolerance adds to reload error. The total transmitter/receiver mismatch and sampling margin determine reliability; "2–3%" is a classroom rule of thumb, not a universal guaranteed limit.

**Worked example: 9600 and 4800 baud at 11.0592 MHz**

For 9600, SMOD=0: $N^{*} = 11059200/(384 \times 9600) = 3$; TH1=$256 - 3 = 253 = \text{FD}_{\text{H}}$. Timer 1 overflows at $921600/3 = 307200$ Hz and the UART divides by 32 to give exactly 9600 baud. For 4800, $N = 6$; TH1=$250 = \text{FA}_{\text{H}}$. For 19200 with SMOD=0, $N^{*} = 1.5$ and no integer count is exact. Set SMOD=1: $N = 3$ and FDH now gives exactly 19200.

| Target baud | SMOD | Count $N$ | TH1 | Error (11.0592 MHz) |
|---|---|---|---|---|
| 1200 | 0 | 24 | E8H | 0% |
| 2400 | 0 | 12 | F4H | 0% |
| 4800 | 0 | 6 | FAH | 0% |
| 9600 | 0 | 3 | FDH | 0% |
| 19200 | 1 | 3 | FDH | 0% |
| 28800 | 0 | 1 | FFH | 0% |
| 57600 | 1 | 1 | FFH | 0% |

The table only establishes hardware baud divisibility; ISR and foreground throughput at higher rates still need a timing budget.

**Worked example: Why a 12 MHz crystal is awkward for 9600**

SMOD=0 gives $N^{*} = 3.255208333$. At $N = 3$ (FDH), $B = 10416.666667$ baud and error $+8.506944\%$. At $N = 4$ (FCH), $B = 7812.5$ baud and error $-18.619792\%$; FDH is closer but still poor.

With SMOD=1, $N^* = 6.510416667$. Compare $N = 6$: $10416.666667$ baud ($+8.506944\%$), and $N = 7$ (F9H): $8928.571429$ baud ($-6.994048\%$). Neither is a good 9600 match. This is a divider/clock problem, not evidence that a 12 MHz 8051 cannot communicate at any baud.

**Why mode 2?** Automatic reload produces periodic overflows without a reload ISR. It is the standard simple design, not the only Timer 1 mode that can feed the UART. Other modes can be used with a justified configuration; software reload changes timing and can introduce jitter. An 8052 can instead use Timer 2, but the Timer 1 formula above does not apply to its special baud-generator mode.

## 2.5 Polling serial programming: complete C51 examples

The initialisation sequence is: stop/configure Timer 1; choose mode 2 with a masked TMOD write; set SMOD; load both TH1 and TL1; set SCON; release the port latches; then start Timer 1. ET1 stays disabled: baud generation is hardware work, not one ISR for every baud-clock overflow.

**Shared header: UART initialisation**

```
#ifndef UART_INIT_H
#define UART_INIT_H
#include <reg51.h>
/* Keil C51; classic 12-clock core; 11.0592 MHz; 9600, 8N1. */
static void uart_init(void)
{
    ES = 0;                 /* Configure before enabling serial IRQ. */
    TR1 = 0;
    ET1 = 0;                /* Timer 1 is a baud clock, not a tick ISR. */
    TMOD = (TMOD & 0x0F) | 0x20;
    PCON &= 0x7F;           /* SMOD = 0; preserve other PCON bits. */
    TH1 = 0xFD;
    TL1 = 0xFD;             /* Correct first overflow as well. */
    SCON = 0x50;            /* Mode 1, REN = 1, TI = RI = 0. */
    P3 |= 0x03;             /* Release RxD/TxD port latches. */
    TR1 = 1;
}
#endif
```

Loading TL1 as well as TH1 gives the intended first overflow. If only TH1 is set after reset, TL1 starts from 00H until the first overflow; subsequent reloads are correct. This is a startup simplification in the classroom, not an incorrect steady-state baud formula.

### Shared header: polling transmission and reception

```
#ifndef UART_POLL_H
#define UART_POLL_H
#include "uart_init.h"
/* ES must remain 0: these functions own TI and RI. */
static void uart_put(unsigned char c)
{
    TI = 0;
    SBUF = c;
    while (TI == 0) { }
    TI = 0;                 /* TI occurs at START of the stop bit. */
}
static unsigned char uart_get(void)
{
    unsigned char c;
    while (RI == 0) { }
    c = SBUF;               /* Read before releasing receive buffer. */
    RI = 0;
    return c;
}
#endif
```

**Transmit ownership.** `uart_put` is called only by foreground code, with no other writer of SBUF. It clears a stale TI before writing, waits for hardware readiness, and clears TI afterwards. It does not guarantee that the final stop interval has ended; this matters if switching off a driver or changing baud immediately after the function returns. Normally the UART itself preserves the required stop interval before a subsequent transmission.

**Receive ordering.** Read SBUF into a local byte *before* clearing RI. With RI still set, the unread holding register cannot be replaced by a later accepted frame. Clearing RI first creates a small window in which another frame can be accepted before the old byte is read. The safer order is also easy to remember: capture, then release.

### Transmit a ready message once

```
#include "uart_poll.h"
void main(void)
{
    unsigned char i;
    const unsigned char code message[] = "8051 READY\r\n";
    IE = 0;
    uart_init();
    for (i = 0; message[i] != 0; ++i)
        uart_put(message[i]);
    while (1) { }
}
```

The `code` array saves data RAM. Each byte goes through the same `uart_put` handshake; the final `while(1)` prevents falling out of the embedded entry function.

**Receive and display an ASCII byte**

```
#include "uart_poll.h"
void main(void)
{
    unsigned char c;
    IE = 0;
    P1 = 0xFF;
    uart_init();
    while (1) {
        c = uart_get();
        P1 = (unsigned char)~c; /* Low-side, active-low LED display. */
    }
}
```

For eight indicators, use individual current-limiting resistors and check the port's total sink-current limit, or add buffers. In this active-low display a received 'A' ($41_H$) lights bit positions 0 and 6, while P1 holds BEH. The classroom's uncomplemented port pattern describes an active-high logical display; the physical LED wiring decides which pattern to use.

**Echo, with ASCII lowercase converted to uppercase**

```
#include "uart_poll.h"
void main(void)
{
    unsigned char c;
    IE = 0;
    uart_init();
    while (1) {
        c = uart_get();
        if (c >= 'a' && c <= 'z') c -= 0x20;
        uart_put(c);         /* Terminal local echo should be off. */
    }
}
```

Subtract 20H only for 'a' through 'z'. Adding 20H converts uppercase to lowercase. Disable terminal local echo to avoid seeing two copies.

**Worked example: A polling receive execution trace**

Initially RI=0, SCON=50H. The terminal sends 'K' (4BH). REN=1 permits reception. Near the accepted stop-bit sample, SBUF receives 4BH and RI becomes 1. The loop exits, C copies 4BH, and then clears RI. A later frame can now be accepted. Writing the same byte to SBUF uses the *transmit* register; it does not erase the received byte.

## 2.6 Receive buffering, lost frames and deadlines

The classic receiver can shift a new character while an old one waits in the holding register. At the final transfer point of a mode-1 frame, acceptance requires RI=0 and either SM2=0 or a valid stop bit. If RI is still 1, the **newly completed frame is discarded**; the old unread byte remains. A plain classic core provides no dedicated overrun counter or FIFO to tell firmware how many frames were lost.

For continuous 9600-baud 8N1 traffic, accepted bytes can complete every 1.041667 ms. Software must capture SBUF and clear RI before the next frame's transfer point. A polling loop that spends 10 ms updating an LCD cannot guarantee this. An ISR can reduce foreground delay, but long interrupt masking, long same/higher-priority ISRs and insufficient software-buffer capacity still cause loss.

An isolated echo has roughly one incoming frame plus one outgoing frame, about 2.083333 ms wire latency from the incoming start to the outgoing end, plus turnaround. That is not a universal maximum input rate of one character every 2 ms: full-duplex RX and TX can overlap, and the holding register provides limited overlap. A blocking echo is still unsuitable as a general guarantee for a busy controller.

**Exam targets and traps**

Be ready to draw the 'A' frame, explain full duplex and SBUF, decode SCON, calculate TH1 with oscillator assumptions, and draw the converter path. Common errors: MSB-first UART drawing; baud treated as useful bytes/s; TI treated as an acknowledgement or end-of-stop indication; missing REN/TR1; RI cleared before SBUF is captured; a TTL adapter mistaken for an RS232 adapter.

Verification and further reading: Classic mode timing and acceptance conditions [INTEL]; independent 80C51 hardware description [NXP]; MAX232 pinout/application [MAX]; prescribed Mazidi chapter map [MAZIDI].

# 3 Interrupts, priority and reliable Embedded C

## 3.1 What an interrupt does, and why it is needed

An **interrupt** requests service for an event while the CPU executes another instruction stream. When the request is enabled and eligible, hardware completes the current instruction and transfers control through a fixed **vector**. An **ISR** captures or services the event, then returns so the interrupted program can resume.

The classroom doorbell analogy is useful: polling repeatedly asks whether someone is at the door; an interrupt lets the controller work until an event requests attention. A flag does not by itself mean the ISR will run: the source, enable bits, priority and current execution state must all be considered.

| Property | Polling | Interrupt service |
|---|---|---|
| Event discovery | Foreground tests a pin/flag. | Hardware requests a vector transfer. |
| CPU while waiting | A blocking wait occupies foreground CPU time. Cooperative polling can still do other work between checks. | Foreground continues until service is accepted; entry/exit and ISR work consume time. |
| Response bound | Depends on interval between polls. | Depends on masking, priority, instruction completion and ISR overhead. |
| Programming cost | Usually simplest for one task/bring-up. | Needs configuration and concurrency reasoning. |
| Data loss | Possible if service is late. | Still possible if service is late or buffers fill. |

An ISR does not literally execute in parallel with main on a single-core 8051. The UART hardware can operate concurrently with the CPU; ISR and foreground instructions take turns. A busy polling loop does not disable enabled interrupts: an ISR can pre-empt the wait even though other foreground work cannot run.

### 3.2 Registers and vector reference

**IE (A8H), bits 7 to 0; classic 8051:**

| EA bit 7 | – bit 6 | – bit 5 | ES bit 4 | ET1 bit 3 | EX1 bit 2 | ET0 bit 1 | EX0 bit 0 |
|---|---|---|---|---|---|---|---|

EA is the global enable; EX0/EX1 enable external sources, ET0/ET1 timer sources, and ES the combined serial source. Both EA and the source enable are required. `IE=90H` enables only serial, `IE=81H` only INT0, and `IE=85H` INT0 plus INT1. A masked update or bit assignment preserves other sources; a complete assignment replaces them. Reserved bits must be kept clear. The 8052 assigns IE.5 to ET2.

**IP (B8H), bits 7 to 0; classic 8051:**

| – bit 7 | – bit 6 | – bit 5 | PS bit 4 | PT1 bit 3 | PX1 bit 2 | PT0 bit 1 | PX0 bit 0 |
|---|---|---|---|---|---|---|---|

For each source, 1 means high priority and 0 low priority. IP selects levels; it does *not* enable interrupts. On an 8052, IP.5 is PT2. Classic hardware has only two levels, even though tie-breaking ranks several sources within each level.

**TCON (88H), bits 7 to 0:**

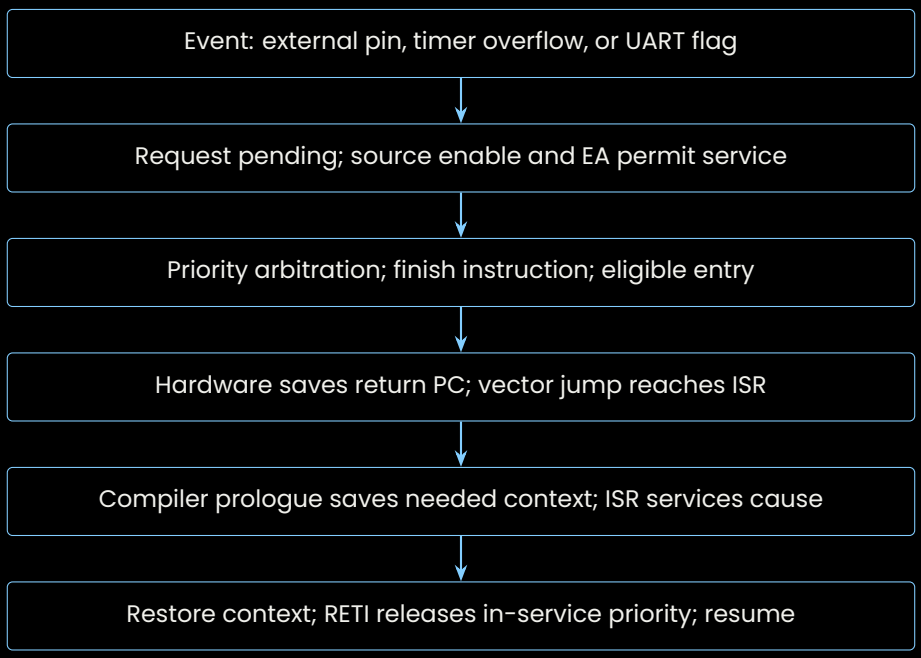
| TF1 bit 7 | TR1 bit 6 | TF0 bit 5 | TR0 bit 4 | IE1 bit 3 | IT1 bit 2 | IE0 bit 1 | IT0 bit 0 |
|---|---|---|---|---|---|---|---|

The lower four bits configure/record external interrupts; the upper four run and record Timer 0/1. IT0/IT1 select falling-edge (1) or active-low level (0) input requests. **IE0** is an event flag in TCON, whereas **EX0** is its enable in IE; the names are easily confused.

| Source | Vector | Flag | Enable | C51 $n$ | Flag clearing |
|---|---|---|---|---|---|
| INT0, P3.2 | 0003H | IE0 | EX0 | 0 | Edge: hardware on vectoring. Level: release the pin. |
| Timer 0 overflow | 000BH | TF0 | ET0 | 1 | Hardware on vectoring; software in polling. |
| INT1, P3.3 | 0013H | IE1 | EX1 | 2 | As for INT0. |
| Timer 1 overflow | 001BH | TF1 | ET1 | 3 | Hardware on vectoring; ET1=0 for UART baud. |
| Serial port | 0023H | RI or TI | ES | 4 | Software; test both causes. |
| Timer 2: **8052 addition** | 002BH | TF2 or EXF2 | ET2 | 5 | Software; 8052 only. |

Reset starts at **0000H**; it is not a sixth maskable interrupt source on the classic 8051. For the normal vector arrangement, Keil interrupt number $n$ corresponds to $0003_{\mathrm{H}} + 8n$. The vector has space for a branch to a larger handler; the compiler/linker builds that connection.

### 3.3 Interrupt entry, return, pending requests and latency




Hardware stacks the return PC; it does not automatically save every register or local variable. In C51, generated ISR entry/exit code protects the registers it needs. The 8051 stack grows upwards in internal RAM; nested ISRs and calls require enough space and correct allocation.

**RETI versus RET.** Both return to a saved address, but RETI additionally releases the interrupt controller's in-service state. A plain RET can leave that state blocked. C51 generates RETI for an `interrupt n` function. Do not call the ISR as an ordinary function or replace its return with inline assembly RET.

**Masking is not event erasure.** A latched edge/timer/serial flag can become set with its enable clear. Enabling service later can cause an already-pending request to run. Multiple events while a one-bit flag is already set may coalesce into one indication. Level requests are different: if the level disappears before acceptance, there may no longer be a request to service. Clearing a flag can intentionally discard a pending event.

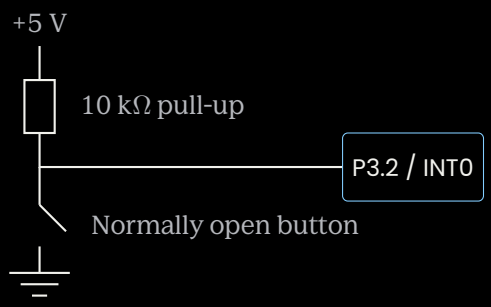
**Latency is not zero.** The classic controller samples and polls requests on machine-cycle boundaries. Instruction completion, an active equal/higher-priority ISR, and certain instructions such as RETI or writes to IE/IP defer entry. At least one instruction can execute between consecutive service entries in the documented cases. Describe "repeated interrupts" rather than literally uninterrupted re-entry with no foreground instruction. A response-time guarantee must include the longest masked interval and blocking ISR, not just vector overhead.

### 3.4 External hardware interrupts and button bounce

INT0 is P3.2; INT1 is P3.3. Both are active-low external inputs with the port latch released to 1. A button can connect the pin to ground, with a pull-up to +5 V defining the released level. Clean pulse sources need suitable level conditioning and a common reference.

| | ITx=1: falling edge | ITx=0: low level |
|---|---|---|
| Request condition | Sampled high-to-low transition latches IEx. | Pin remains low as a request condition. |
| Holding the input low | No new edge merely from holding it. | Repeated service if low remains asserted after return. |
| Flag clearing | Edge flag clears in hardware on vectoring. | Requesting device must release its low condition in response to service. |
| Typical use | Debounced switch; conditioned pulse. | Device holds IRQ low until firmware handles/acknowledges it. |

External pins are sampled rather than asynchronous pulse counters of unlimited speed. For a reliably detected falling edge, both the high and low intervals must span the documented sampling requirement (at least one machine cycle on the classic description); provide margin for real signals. A short glitch may be missed; pulse conditioning matters.




**Bounce.** Mechanical contacts can create several falling edges in one physical press. Selecting IT0=1 fixes the held-low level problem, but does not debounce the switch. Hardware RC filtering with a Schmitt input or a timed firmware debounce is needed if one press must count once.

### Count clean pulses, with a safe 16-bit snapshot

```
#include <reg51.h>
volatile unsigned int pulses = 0;
void int0_isr(void) interrupt 0
{
    ++pulses;               /* Clean, conditioned pulses; not raw button. */
}
void main(void)
{
    unsigned int snapshot;
    bit saved_ea;
    IE = 0;
    P3 |= 0x04;             /* P3.2 input latch must be 1. */
    IT0 = 1;
    IE0 = 0;
    EX0 = 1;
    EA = 1;
    while (1) {
        saved_ea = EA;
        EA = 0;
        snapshot = pulses;  /* Atomic snapshot of a 16-bit counter. */
        EA = saved_ea;
        P1 = (unsigned char)~snapshot;
        P2 = (unsigned char)~(snapshot >> 8);
    }
}
```

This is appropriate for a conditioned input, not an unfiltered button. A count can wrap modulo 65536. Both output bytes come from one atomic snapshot; otherwise an interrupt between byte reads can produce a value that never existed.

**A button ISR with foreground debounce**

```
#include "timer_delay.h"
sbit BUTTON = P3^2;
volatile bit press_pending = 0;
void int0_isr(void) interrupt 0
{
    EX0 = 0;                /* Suppress further bounce interrupts. */
    press_pending = 1;
}
void main(void)
{
    unsigned char count = 0;
    IE = 0;
    BUTTON = 1;             /* External 10k pull-up; switch to GND. */
    P1 = 0xFF;
    IT0 = 1;
    IE0 = 0;
    EX0 = 1;
    EA = 1;
    while (1) {
        if (press_pending) {
            press_pending = 0; /* EX0 is still disabled. */
            delay_ms(20);
            if (BUTTON == 0) {
                ++count;
                P1 = (unsigned char)~count;
                do {
                    while (BUTTON == 0) { }
                    delay_ms(20); /* Require a stable release. */
                } while (BUTTON == 0);
            }
            IE0 = 0;        /* Discard edges during debounce window. */
            EX0 = 1;
        }
    }
}
```

The ISR disables EX0 and posts a flag. Main waits about 20 ms, confirms the low state, counts once, then requires a release lasting about 20 ms before clearing any bounce-latched IE0 and re-enabling EX0. The debounce time is an engineering assumption for this example, not a universal switch specification. Main blocks during a held press, but the ISR stays short; a nonblocking timer/state machine is preferable in a system with other deadlines.

To adapt the clean-input example to INT1, use P3.3, IT1/IE1/EX1 and `interrupt 2`. Setting `interrupt 1` would install a Timer 0 handler, not INT1.

## 3.5 Priority, nesting and simultaneous requests

Apply these rules in order:

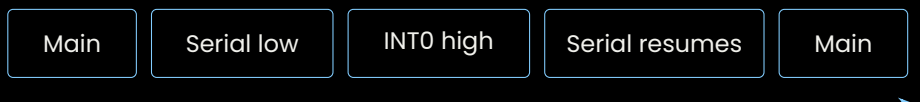
1. An eligible high-priority request outranks an eligible low-priority request.
2. A high-priority ISR may pre-empt a low-priority ISR. An equal- or lower-priority request cannot pre-empt the active ISR. No classic maskable interrupt pre-empts an active high-priority ISR.

3. If eligible requests of the *same* level are pending at arbitration, the fixed polling order is INT0 → Timer 0 → INT1 → Timer 1 → Serial. Timer 2, if present on an 8052, follows serial.

The tie order does not create additional nesting levels and is not an arrival-time queue. A timer or edge flag records a pending condition, not an unlimited number of events. A higher-priority request that arrives later can be served earlier.

**Worked example: Three cases that distinguish priority from tie order**

**Case 1: IP=00H, INT0 and serial pending together.** Both are low. INT0 wins the fixed order; serial remains pending and is handled when eligible after the INT0 handler.
**Case 2: IP=10H, the same pending pair.** PS=1 makes serial high; PX0=0 leaves INT0 low. Serial wins even though its vector address is larger.
**Case 3: IP=01H, serial ISR running, then INT0 arrives.** Serial is low and INT0 high. Serial is suspended, INT0 runs, then serial resumes before main. If IP=00H instead, INT0 must wait for the serial ISR even though INT0 has the first tie rank.



Time →; only the high ISR nests inside the low ISR

## 3.6 Keil C51 ISR syntax and shared data

```
void serial_isr(void) interrupt 4
{
    /* Called through vector 0023H; returns with RETI. */
}
```

An ISR has no parameters and returns no value. C51's `interrupt n` selects the vector; it is not ISO C. The optional `using 1` selects register bank 1; it does not select interrupt priority. A sensible bank arrangement reserves bank 0 for foreground, bank 1 for low-priority handlers and bank 2 for high-priority handlers. Same-level handlers do not nest, but a high handler must not clobber the bank of a suspended low handler. Without `using`, the compiler saves/restores the registers needed; it still requires stack space.

**Reentrancy.** C51 normally allocates function arguments/local storage in fixed locations and may overlay storage for functions that cannot be active together. Calling the same ordinary function from main and an ISR can corrupt that storage. `reentrant` is a separate C51 attribute requiring appropriate simulated-stack support, and calls across different `using` banks require the documented conventions. These example ISRs call no ordinary helper functions; formatting, LCD writes and command interpretation stay in main.

**Volatile is necessary but not sufficient.** A shared object such as `volatile unsigned int pulses` tells the compiler that reads/writes have observable asynchronous significance. It does not make a 16-bit access indivisible on an 8-bit CPU, protect a read-modify-write increment, reserve a queue slot or turn a one-bit flag into an event counter.

**Worked example: A torn read and a lost notification**

Suppose a counter changes from 00FFH to 0100H. An unprotected foreground read can combine an old low byte FFH with a new high byte 01H, yielding 01FFH (511), or another inconsis-

tent mixture depending on read order. A short EA-masked snapshot prevents that interleaving.
For `if(got) { handle(ch); got=0; }`, a new ISR can set `got=1` while `handle` runs; the subsequent foreground clear erases its notification. Copy the pending data and clear its flag *together* under a brief interrupt mask, then process the local copy after restoring the original EA value. A single-byte mailbox must also define what happens if another byte arrives before it is emptied.

Restoring a saved EA value is better than blindly writing EA=1 at the end of a critical section: the caller may have entered with interrupts already disabled. Keep the mask interval short enough to meet peripheral deadlines. A disabled interrupt does not stop the UART or timer hardware.

### 3.7 The serial interrupt: two flags, one vector

The serial request is RI OR TI, gated by ES and EA. Both flags can be set together. Test them with two independent `if` statements; an `else if` can leave the second cause pending until another entry. Read SBUF before clearing RI. Clear TI when acknowledging its transmit-ready event. Clearing only RI does not make an active TI request disappear.

**The polling-transmitter / ISR conflict**

If main executes `while(TI==0)` while an enabled serial ISR clears TI, the ISR can clear the flag before main observes it. Main can then wait indefinitely. Give the flag one owner. Either leave ES=0 for polling serial, or let the ISR acknowledge TI and publish a separate `volatile` software-ready flag that main waits on. Masking all interrupts for a complete transmitted string would sacrifice receive deadlines.

### 3.8 A complete command interface with a receive queue

This retains the classroom commands '1', '2', '0' and 'S', with a short reply representing LED state: '0' through '3'. '1' enables LED 1, '2' enables LED 2, '0' clears both, and 'S' reports without changing them; unknown input returns '?'. The physical LEDs are active-low, each +5 V → resistor → LED → P1 pin. The UART uses the MAX232 connection already described.

The ISR captures accepted bytes into a 16-byte ring; one slot is reserved to distinguish full from empty, so **15 unread bytes** fit. Main removes bytes inside a short critical section and interprets a local copy. On full, the newest incoming software byte is discarded and `dropped` saturates at 255; previously queued bytes stay ordered. This counter detects *software queue overflow*, not frames discarded inside the UART before the ISR can capture them.

### Buffered RX and ISR-owned transmit flag (part 1 of 2)

```
#include "uart_init.h"
#define RX_MASK 15
volatile unsigned char rx[16];
volatile unsigned char head = 0, tail = 0;
volatile unsigned char dropped = 0;
volatile bit tx_ready = 1;
void serial_isr(void) interrupt 4
{
    unsigned char b, next;
    if (RI) {
        b = SBUF;
        RI = 0;
        next = (head + 1) & RX_MASK;
        if (next != tail) {
            rx[head] = b;
            head = next;
        } else if (dropped != 255) ++dropped;
    }
    if (TI) {
        TI = 0;             /* Only the ISR owns TI. */
        tx_ready = 1;
    }
}
static bit rx_get(unsigned char *out)
{
    bit saved_ea, available;
    saved_ea = EA;
    EA = 0;
    available = (head != tail);
    if (available) {
        *out = rx[tail];
        tail = (tail + 1) & RX_MASK;
    }
    EA = saved_ea;
    return available;
}
```

**Buffered RX and ISR-owned transmit flag (continued: part 2 of 2)**

```
static void irq_put(unsigned char c)
{
    bit saved_ea;
    while (tx_ready == 0) { } /* Wait for SOFTWARE notification. */
    saved_ea = EA;
    EA = 0;
    tx_ready = 0;
    SBUF = c;
    EA = saved_ea;
}
void main(void)
{
    unsigned char c, leds = 0;
    IE = 0;
    uart_init();
    P1 = 0xFF;              /* Two active-low LEDs, P1.0/P1.1. */
    ES = 1;
    EA = 1;
    while (1) {
        if (rx_get(&c)) {
            switch (c) {
            case '1': leds |= 1; break;
            case '2': leds |= 2; break;
            case '0': leds = 0; break;
            case 'S': break;
            default: irq_put('?'); continue;
            }
            P1 = 0xFC | ((~leds) & 3);
            irq_put('0' + leds); /* Reply encodes state 0..3. */
        }
        /* Other SHORT foreground work can run here. */
    }
}
```

**Initialisation.** The globals begin at zero, except `tx_ready=1` because the transmitter starts idle. `uart_init` clears TI/RI and keeps ES disabled until the data structures and output pins are ready. Main then sets ES and EA. No ISR waits for a transmitted byte.

**RI path.** The ISR copies SBUF immediately, clears RI, calculates the next head, and either publishes the byte or records a drop. `head==tail` means empty; `(head+1)&15==tail` means full. Modulo indices always remain 0–15.

**TI path.** Only the ISR clears hardware TI and sets `tx_ready`. `irq_put` waits on this *software flag*, then atomically marks the transmitter busy and writes SBUF. It must be called from main with the serial interrupt able to run; calling it with EA=0 or from the serial ISR risks deadlock. Its reply is one byte, so foreground wait remains bounded by the normal transmit-ready interval.

**Worked example: Queue trace and a finite buffering budget**

Starting head=tail=0, receive ‘1’: rx[0]=31H, head=1. Receive ‘S’: rx[1]=53H, head=2. Main removes ‘1’, advances tail to 1 and lights LED 1. The remaining ‘S’ stays queued even while the reply is sent. A later removal yields ‘S’, not an overwritten command.

At a continuous 960 bytes/s, an empty 15-byte queue provides only $15/960 = 15.625$ ms of nominal buffering time. That is not permission for unbounded main-loop work: once its capacity is occupied, additional software arrivals are dropped. A loss-free system needs bounded service, enough buffering and possibly flow control or a command/acknowledgement protocol.

## 3.9 A nonblocking transmit queue: correcting the advanced exercise

The classroom's advanced ring-transmitter idea is valuable, but a real implementation needs full detection and an atomic idle kick. Main below attempts to enqueue one byte and immediately receives success/failure. The ISR advances transmission. Its RI path intentionally discards received bytes because this is a transmit-only demonstration; it is not a receive example.

**Bounded TX queue with a race-free start (part 1 of 2)**

```
#include "uart_init.h"
#define MASK 15
volatile unsigned char tx[16];
volatile unsigned char head = 0, tail = 0;
volatile bit busy = 0;
/* Main is the only caller. Return 0 means FULL: caller retries. */
static bit tx_try_put(unsigned char c)
{
    unsigned char next;
    bit saved_ea;
    saved_ea = EA;
    EA = 0;
    next = (head + 1) & MASK;
    if (next == tail) {
        EA = saved_ea;
        return 0;
    }
    tx[head] = c;
    head = next;
    if (!busy) {
        busy = 1;
        TI = 0;             /* Controlled kick-off, inside critical section. */
        SBUF = tx[tail];
        tail = (tail + 1) & MASK;
    }
    EA = saved_ea;
    return 1;
}
```

**Bounded TX queue with a race-free start (continued: part 2 of 2)**

```
void serial_isr(void) interrupt 4
{
    if (RI) {
        (void)SBUF;         /* This TX-only demo intentionally discards RX. */
        RI = 0;
    }
    if (TI) {
        TI = 0;
        if (head != tail) {
            SBUF = tx[tail];
            tail = (tail + 1) & MASK;
        } else busy = 0;
    }
}
void main(void)
{
    IE = 0;
    uart_init();
    ES = 1;
    EA = 1;
    while (1) {
        if (tx_try_put('A')) {
            /* An 'A' was accepted, not necessarily fully sent yet. */
        }
        /* Other foreground work: no hardware-TI wait loop. */
    }
}
```

**Why the critical section matters.** Without it, main can observe “busy”, the ISR can then empty the queue and clear busy, and main can publish a new byte without starting transmission. The new byte remains stranded. Enqueuing and checking/kicking idle under one brief mask prevents this. Capacity is 15 queued bytes plus, when busy, a byte in the hardware transmit path. A full return is a policy result, not a silently overwritten older byte.

## 3.10 Timing and design questions the exam can ask

**What is the vector and C51 number for INT1?**

**Model answer.** 0013H and `interrupt 2`. Its enable is EX1 and trigger selector IT1; EA must also be set.

**Does IP=01H enable INT0?**

**Model answer.** No. It sets PX0 to high priority. EX0 and EA in IE are still needed.

**Can a high serial ISR be interrupted by low INT0?**

**Model answer.** No. INT0’s tie rank does not override the active high-priority level. A latched INT0 request waits until eligible.

**Why not put LCD output and string transmission in an ISR?**

**Model answer.** They increase blocking time, can lose UART frames and can invoke non-reentrant helpers or deadlock on a flag owned by that ISR. Capture briefly, then process in main.

Verification and further reading: Classic interrupt arbitration and flag handling [INTEL], [NXP]; official vector/function convention [KEILIRQ]; register-bank and reentrancy rules [KEILBANK], [KEILREENT]; university teaching context [UGC].

# 4 LCD interfacing

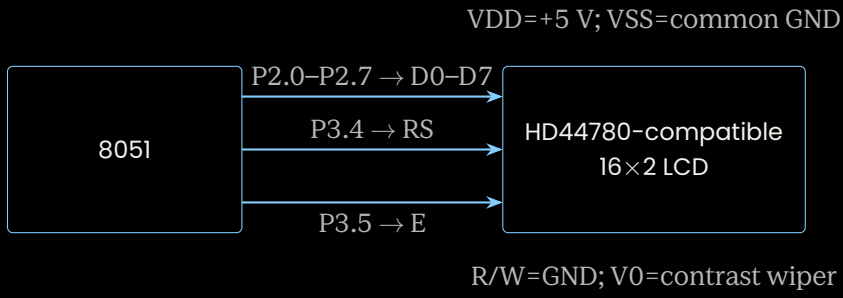
## 4.1 What the LCD interface controls

An LCD shows information without a PC connection. The selected component is a **5 V, 16×2 character LCD compatible with the Hitachi HD44780U**. Its controller stores character codes and generates the dot pattern; the 8051 sends commands and characters, not individual liquid-crystal drive waveforms. A character module is not the same interface as a graphics LCD or an $I^2C$ backpack.

The controller's instruction register accepts commands; its data register transfers character codes into display data RAM (DDRAM). The **RS** pin chooses which register, **R/W** chooses read or write, and **E** is the enable strobe. Character generator ROM contains the built-in glyphs; CGRAM can hold custom patterns.

| Module pin | Selected connection | Reason |
|---|---|---|
| 1 VSS; 2 VDD | Ground; +5 V | Controller power, with nearby bypassing. |
| 3 V0 | Wiper of a 10 kΩ potentiometer between +5 V and ground | Adjust contrast; a powered display can appear blank if this is wrong. |
| 4 RS | P3.4 | 0: command; 1: character data. |
| 5 R/W | Ground | Write-only example; use timed waits rather than busy reads. |
| 6 E | P3.5 | Pulse high, then low to commit a write. |
| 7–14 D0–D7 | P2.0–P2.7 | Eight-bit data bus, bit order preserved. |
| 15/16, if fitted | Backlight supply as module specifies | Use its rated current/resistor; GPIO does not power the backlight. |

P3.4/P3.5 can also be external counter inputs, but this program uses them as GPIO, not counters. P2 is not being used as an external-memory address bus. All logic grounds are common.



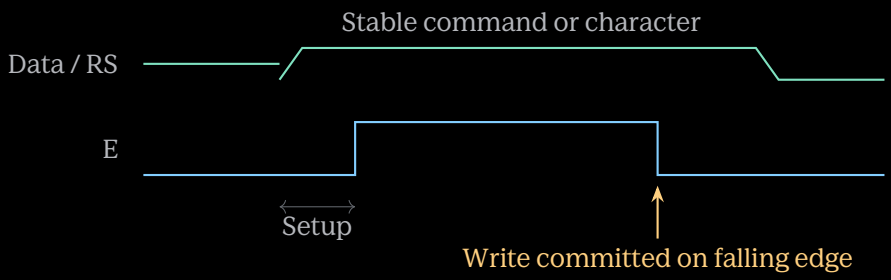
## 4.2 Commands, addresses and write timing

| Command | Meaning in the selected example |
|---|---|
| 30H | Force/select 8-bit interface in the power-up recovery sequence. |
| 38H | Function set: DL=1 (8-bit), N=1 (two lines), F=0 (5×8 font). |
| 08H / 0CH | Display off / display on with cursor and blink off. |
| 01H / 02H | Clear display / return home; these take longer than normal writes. |
| 06H | Entry mode: increment address, no automatic display shift. |
| 80H | Set DDRAM address 00H: start of first line. |
| C0H | Set DDRAM address 40H: start of second line. |
| 80H + address | General DDRAM-address command; the command's high bit selects address setting. |

On the common 16×2 mapping, the visible first-line addresses are 00H–0FH and the second-line addresses 40H–4FH. The second line does not start at 10H just because the module has sixteen columns. For column 5 counting from 1 on the second line, address=44H and command=C4H.

For a write, establish RS and data with E low, raise E for the minimum pulse width, lower E, and allow the instruction to complete before the next one. The HD44780U 5 V timing specifies setup/hold times and an enable pulse of at least 230 ns in its 4.5–5.5 V conditions; the more conservative 450 ns requirement appears for lower supply conditions. The example exceeds both with 8051 machine-cycle NOPs.

At the reference internal oscillator of 270 kHz, most commands/data writes take at most 37 $\mu$s, while clear/home take 1.52 ms. Execution time scales with the controller clock. A routine that always waits only 50 $\mu$s will fail after clear/home. Here every write waits conservatively for about 3 ms; it is deliberately simple and slower than necessary.



## 4.3 Complete eight-bit output program

### Initialise and display two lines

```
#include <intrins.h>
#include "timer_delay.h"
sbit LCD_RS = P3^4;
sbit LCD_E  = P3^5;
/* HD44780-compatible 16x2; D0..D7=P2; R/W permanently GND. */
static void lcd_write(unsigned char b, bit is_data)
{
    LCD_E = 0;
    LCD_RS = is_data;
    P2 = b;
    _nop_();                /* Data/control setup > 1 machine cycle. */
    LCD_E = 1;
    _nop_();
    _nop_();                /* Enable pulse comfortably > 450 ns. */
    LCD_E = 0;
    delay_ms(3);            /* Conservative command AND clear delay. */
}
static void lcd_init(void)
{
    LCD_RS = 0;
    LCD_E = 0;
    delay_ms(20);
    lcd_write(0x30, 0);      /* Force 8-bit state: first wake-up. */
    delay_ms(5);
    lcd_write(0x30, 0);
    delay_ms(1);
    lcd_write(0x30, 0);
    lcd_write(0x38, 0);      /* 8-bit, two lines, 5x8 font. */
    lcd_write(0x08, 0);      /* Display off during initialisation. */
    lcd_write(0x01, 0);      /* Clear DDRAM. */
    lcd_write(0x06, 0);      /* Increment address, no display shift. */
    lcd_write(0x0C, 0);      /* Display on, cursor/blink off. */
}
void main(void)
{
    unsigned char i;
    const unsigned char code line[] = "MODULE 3 READY";
    IE = 0;
    lcd_init();
    lcd_write(0x80, 0);      /* First line, first column. */
    for (i = 0; line[i] != 0; ++i) lcd_write(line[i], 1);
    lcd_write(0xC0, 0);      /* Second line, first column. */
    lcd_write('8', 1);
    lcd_write('0', 1);
    lcd_write('5', 1);
    lcd_write('1', 1);
    while (1) { }
}
```

This program includes `timer_delay.h` and Keil's `intrins.h` for `_nop_()`. The three 30H writes implement recovery to 8-bit operation even if the internal power-on reset sequence cannot be assumed. After the first, the total wait exceeds 4.1 ms; after the second it exceeds 100 $\mu$s. Start the initial 20 ms wait only after the supply is valid; uncontrolled slow power ramps need circuit-specific validation.

Expected output: first line "MODULE 3 READY", second line "8051". RS is 0 for 80H/C0H and 1 for character writes. ASCII '8' (38H) as data has a different meaning from command 38H because RS differs.

### 4.4 Four-bit operation and busy-flag reading

Four-bit interfacing saves four GPIO pins. Connect D4–D7, send the *high nibble first* and pulse E, then send the low nibble and pulse E again. The power-up procedure needs the documented 3,3,3,2 nibble sequence to establish 4-bit operation; after that, complete two-nibble instructions such as 28H are used. Do not simply remove four wires from the eight-bit program and keep its byte writes.

If R/W is controlled instead of tied low, busy flag DB7 can be read with RS=0, R/W=1. Release the MCU data-port latches to inputs before the LCD drives them, pulse E and sample DB7, then restore write direction. In 4-bit mode, read both nibbles to keep the interface aligned. Busy checking cannot replace the initial power-up recovery waits before the controller is in a known state.

**Explain LCD command/data selection in 3 marks.**

**Model answer.** RS=0 selects an instruction and RS=1 selects character data. R/W=0 writes; R/W=1 reads. A valid enable strobe and completion wait are required for each operation.

**Common errors and longer-question structure**

For a 9-mark interface answer, identify the module, draw its power/contrast/data/control connections, explain commands and timing, and give initialisation plus character-output code. Common faults: no contrast connection; swapped data bits; R/W left floating; E held high rather than pulsed; treating a 4-bit bus as an 8-bit bus; short delay after clear; wrong second-line address.

Verification and further reading: HD44780U pin functions, instructions, bus timing and "Initializing by Instruction" [LCD]; Mazidi LCD chapter/section map [MAZIDI].

## 5 ADC and DAC interfacing

### 5.1 Analogue quantities and digital codes

An **ADC** maps an analogue input to a binary code so firmware can measure it. A **DAC** maps a digital code to an analogue current or voltage so firmware can generate a reference, waveform or control signal. Sampling, quantisation and electrical conversion are different operations: a faster CPU does not automatically make an external converter faster or more accurate.

For an $n$-bit converter there are $2^n$ distinct codes, numbered 0 through $2^n - 1$. For an input span $V_{\text{span}}$ divided into $2^n$ equal steps, one ideal LSB is

$$q = \frac{V_{\text{span}}}{2^n}.$$

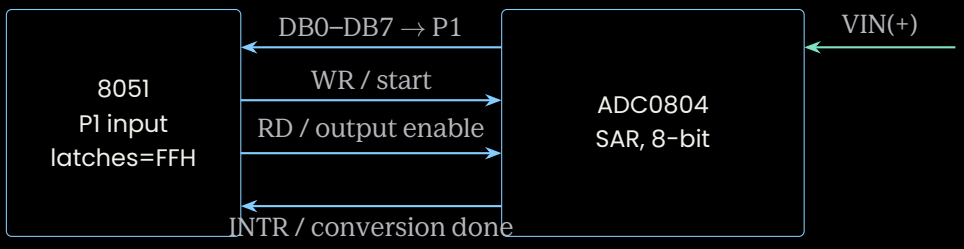
Resolution is this step size; accuracy includes offset, gain, nonlinearity, reference error and noise. Do not replace $2^n$ with $2^n - 1$ without stating a different endpoint convention.

### 5.2 ADC0804: selected device and handshake

The selected **ADC0804-N** is an 8-bit successive-approximation (SAR) converter with a parallel tri-state output. It tests the most significant decision first and refines the code through comparisons. Its external or RC-generated clock runs the conversion; the 8051 controls start/read and watches conversion complete.

For the standard unipolar example, VIN(−)=0 and VIN(+)=the measured voltage. The nominal full-scale span is $2V_{\mathrm{REF}/2}$. Leaving the reference pin unconnected gives the nominal supply-related span; driving VREF/2 with a stable 2.5 V gives a 5 V span. A smaller reference reduces span and LSB size, but does not permit an input beyond the device's supply/common-mode limits.

| ADC0804 20-pin connection | Selected circuit |
|---|---|
| CS (1) | Ground: this is the only device on the GPIO bus. |
| RD (2) / WR (3) | P3.7 / P3.6 as GPIO. Both active-low. No external MOVX bus. |
| INTR (5) | P3.2 as an input; EX0 is disabled in this polling example. |
| DB0 (18) through DB7 (11) | P1.0 through P1.7, preserving binary significance. |
| VIN+ (6), VIN− (7) | Signal input, ground. Signal source within supply limits. |
| AGND (8), DGND (10) | Common ground with sensible analogue/digital return routing. |
| VREF/2 (9), VCC (20) | Stable 2.5 V reference; +5 V supply with bypassing. |
| CLK IN (4), CLK R (19) | 10 kΩ between CLK R and CLK IN; 150 pF from CLK IN to ground, or a documented external clock. |




CS=0; VIN(−)=0; +5 V, reference, clock and grounds

**Sequence.** Hold RD high so outputs are disabled. With CS low, pulse WR low then high; the rising edge starts conversion after the reset portion. Wait for INTR to go low. Assert RD low to enable the output data, allow the specified access time, copy the port, then return RD high to tri-state the outputs. INTR means end-of-conversion; it is not an 8051 interrupt unless connected and configured as one.

The chosen RC clock is nominally

$$f_{\mathrm{CLK}} \simeq \frac{1}{1.1RC} = \frac{1}{1.1(10000)(150 \times 10^{-12})} = 606060.606\ \mathrm{Hz}.$$

The 64-cycle SAR conversion count corresponds to approximately 105.6 $\mu$s at that nominal clock; the actual timing includes device behaviour and R/C tolerance. At the datasheet's 640 kHz reference, typical conversion is about 100 $\mu$s. Read the selected speed/accuracy limits rather than assuming RC values are exact.

**Bounded conversion wait, then read the data bus**

```c
#include <intrins.h>
#include "timer_delay.h"
sbit ADC_WR   = P3^6;
sbit ADC_RD   = P3^7;
sbit ADC_INTR = P3^2;
/* CS=GND, DB0..DB7=P1; no external memory bus is used. */
static bit adc_read(unsigned char *out)
{
    unsigned int timeout = 60000;
    ADC_RD = 1;
    P1 = 0xFF;              /* Release quasi-bidirectional data pins. */
    ADC_WR = 0;
    _nop_();
    _nop_();
    ADC_WR = 1;             /* Rising edge starts conversion. */
    while (ADC_INTR && --timeout) { }
    if (ADC_INTR) return 0;  /* Loop limit; not a calibrated timeout. */
    delay_ms(1);            /* Includes >8 ADC clocks before RD. */
    ADC_RD = 0;
    _nop_();
    _nop_();                /* Allow output-access time. */
    *out = P1;
    ADC_RD = 1;             /* Tri-state the ADC data outputs. */
    return 1;
}
void main(void)
{
    unsigned char sample;
    IE = 0;                 /* P3.2 is polled, EX0 is disabled. */
    ADC_WR = 1;
    ADC_RD = 1;
    ADC_INTR = 1;
    P2 = 0xFF;
    while (1) {
        if (adc_read(&sample))
            P2 = (unsigned char)~sample;
        else P2 = 0xFF;     /* Off pattern on timeout; no false sample. */
    }
}
```

The loop-count timeout prevents an infinite wait if a clock or connection is missing; it is not an exact millisecond timeout. The deliberate delay after INTR ensures more than eight ADC clocks before the read strobe, accommodating the datasheet's INTR-reset timing note. This sacrifices sample rate for clarity. P1 is released to FFH before the ADC drives it; P2 displays the captured code with active-low indicators. After an error the display goes off rather than showing an invented new measurement.

**Worked example: Resolution, code and reference scaling**

With an 8-bit ADC and 5 V span, $q = 5/256 = 0.01953125$ V $= 19.53125$ mV. For an ideal ADC0804-style code-centre convention, the estimated voltage for code $D$ is $Dq$. A 2.000

V input corresponds to $2/0.01953125 = 102.4$ LSB; nearest code is 102=66H, representing 1.9921875 V. The difference is 7.8125 mV, within half an ideal LSB.
Code 200=C8H represents $200q = 3.90625$ V. Code FFH represents a nominal centre of $255q = 4.98046875$ V, not an extra code 256 at 5 V. Endpoint saturation limits the first/last bins.
If VREF/2=1.28 V, span=2.56 V and $q = 10$ mV. A nominal 1.23 V input maps to code 123=7BH; a code of 180 represents 1.80 V. The analogue input must stay within this span for useful unsaturated measurement.

Ideal nearest-centre quantisation has about $\pm q/2$ error away from saturation; the ADC0804's specified converter errors and reference errors are additional real considerations. At exact transition boundaries the selected transfer characteristic and nonideal errors decide the code. A floor-bin model is a valid educational convention if labelled, but it has different centre/transition interpretation.

### 5.3 DAC0808: current output and voltage conversion

The selected **DAC0808** is an 8-bit current-output DAC. A weighted ladder/current-switch network converts the unsigned input into output current. It is not a self-contained 0–5 V voltage-output peripheral and does not need a start-conversion handshake for each parallel code.

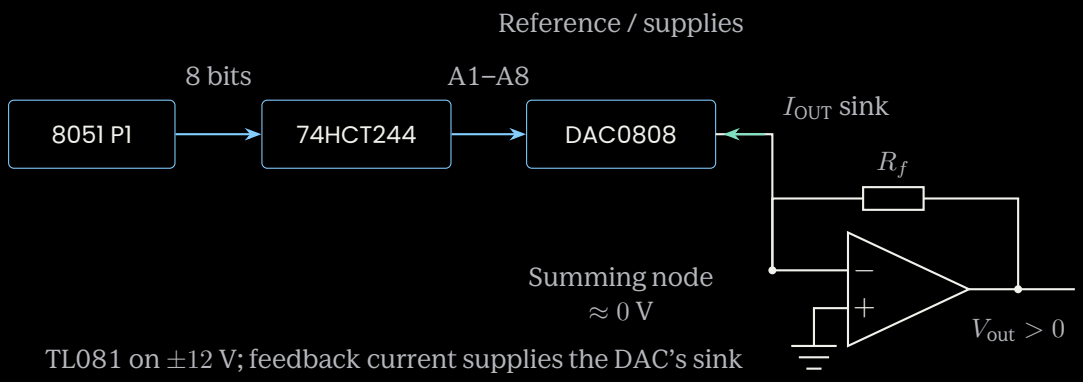
For code $D$ and reference current $I_{\text{REF}}$,

$$I_{\text{OUT}} = I_{\text{REF}} \frac{D}{256}, \qquad I_{\text{FS}} = I_{\text{REF}} \frac{255}{256}.$$

Full-scale code is 255, so its output is one LSB below the ideal reference-current endpoint. The current direction and output-voltage compliance must be respected when designing the load or current-to-voltage amplifier.

| DAC0808 DIP connection | Selected circuit assumption |
|---|---|
| A1 (5) through A8 (12) | A1 is MSB. P1.7 $\rightarrow$ A1, . . ., P1.0 $\rightarrow$ A8, through a non-inverting 74HCT244 buffer. Enable both buffer groups (active-low OE pins=0). |
| VCC (13), VEE (3), GND (2) | +5 V, $-12$ V, common ground, with bypassing. This DAC requires a negative supply in this selected configuration. |
| VREF+ (14), VREF$-$ (15) | Dedicated stable +5 V reference through $R_{14} = 2.5$ k$\Omega$; matching $R_{15} = 2.5$ k$\Omega$ to ground. Nominal $I_{\text{REF}} = 2$ mA. |
| COMP (16) | 47 pF to VEE in this reference configuration, exceeding the datasheet's 37 pF minimum for 2.5 k$\Omega$. |
| IOUT (4) | Current-sinking output to an op-amp summing node held near ground; observe compliance. |

The 74HCT244 provides defined bus drive with TTL-compatible inputs from the classic MCU. Its channels do not invert. Unused logic inputs are tied to defined levels. Buffer and MCU use +5 V and common ground.




The drawing specifies signal/current direction rather than every power pin. Use a suitable op-amp, for example a TL081 on $\pm 12$ V, whose common-mode and output-swing limits accommodate 0–5 V. Its inverting input is the node and its non-inverting input is grounded. Because this selected DAC *sinks* current from the node, KCL gives

$$\frac{V_{\text{out}} - 0}{R_f} = I_{\text{OUT}}, \quad V_{\text{out}} = +R_f I_{\text{OUT}}.$$

If a different DAC/current-source topology injects current *into* an inverting node, the voltage sign reverses. State the current arrow and circuit before quoting a signed DAC formula. Do not use a large resistor from IOUT to ground that drives the pin outside its compliance range.

**Worked example: DAC current and output levels**

$I_{\text{REF}} = 5\text{ V}/2.5\text{ k}\Omega = 2$ mA, and choose $R_f = 2.5\text{ k}\Omega$. The current LSB is $2/256 = 7.8125\ \mu$A; the voltage LSB is $19.53125$ mV. For 80H ($D = 128$), $I_{\text{OUT}} = 1$ mA and $V_{\text{out}} = +2.5$ V. For FFH, $I_{\text{OUT}} = 1.9921875$ mA and $V_{\text{out}} = +4.98046875$ V. Code 00H ideally gives zero, with real zero-scale error.

**Generate a slowly rising sawtooth code sequence**

```
#include "timer_delay.h"
/* DAC0808: P1.7->A1(MSB), ... P1.0->A8(LSB), via HCT244. */
void main(void)
{
    unsigned char sample = 0;
    IE = 0;
    P1 = 0;
    while (1) {
        P1 = sample;
        delay_ms(1);        /* Nominal dwell; includes software overhead. */
        ++sample;           /* Unsigned 8-bit wrap, 255 -> 0. */
    }
}
```

The unsigned byte increases through 0–255 and wraps to 0. The output climbs in small steps and then drops. With an *ideal exact* 1 ms update interval, one cycle has 256 updates, period 256 ms and frequency 3.90625 Hz. The actual blocking-delay implementation includes overhead and has a slightly longer period; no exact oscilloscope frequency is claimed. For a prescribed waveform frequency, schedule updates from a verified timer and allow the DAC and amplifier to settle. A lookup table can generate a sine approximation.

**Exam targets and common errors**

Draw ADC start/wait/read timing and distinguish INTR from RI; calculate LSB and code; explain SAR and reference scaling. For DAC, state MSB/LSB mapping, supplies, reference-current and I/V stages, then calculate output. Errors include 255 used in a 256-step transfer equation, reference pin voltage mistaken for ADC span, P1 left driving zero during ADC reads, and a DAC0808 output assumed to be a directly available 5 V voltage.

Verification and further reading: ADC0804 control, transfer and clock details [ADC]; DAC0808 reference, compensation and compliance [DAC]; bus buffer [HCT244]; selected op-amp limits [OPAMP].

# 6 Relay and motor interfacing

## 6.1 Logic control versus load power

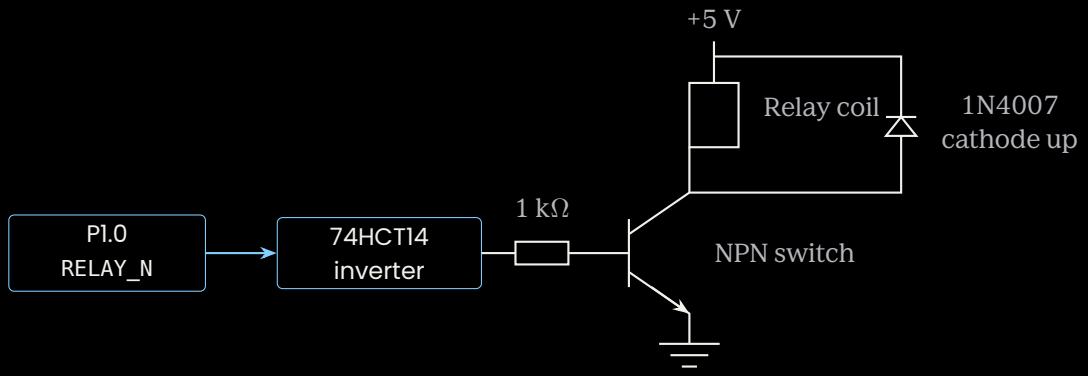
A microcontroller pin communicates a logic decision; it is not the power supply for an actuator. Coils and motors need more current than classic GPIO can provide, and their inductance produces voltage transients when current changes. The interface therefore includes a **driver**, a suitable load supply, common logic ground where required, decoupling and an inductive-current return path.

Classic P1/P2/P3 pull-ups are weak for sustained source current. A ULN2003A input may need roughly a milliampere to drive a significant collector current; simply declaring the MCU output "5 V high" does not prove it can supply that input current. The examples use **74HCT14** inverters with TTL-compatible, low-current inputs and stronger outputs. A GPIO reset value of 1 then becomes driver input 0, keeping the load off. This explicit inversion must also appear in code.

## 6.2 Relay: contacts, transistor driver and flyback

A **relay** uses an electromagnet to move contacts. The coil-control circuit and the contact load circuit are distinct. **COM** is the common contact, **NO** is open when the coil is unenergised, and **NC** is closed when unenergised. Energising the selected SPDT relay connects COM to NO. Coil voltage/current and contact voltage/current are different ratings.

The selected example uses an **Omron G5V-1 DC5** signal relay (5 V coil, nominal 167 Ω, approximately 30 mA), a **2N2222A-class NPN** low-side transistor, a **1 kΩ base resistor**, a **1N4007** flyback diode and a **74HCT14** inverter. Confirm the actual transistor package's emitter/base/collector pinout; different parts sold under related names have different pin arrangements.




Logic supply=+5 V; emitter and logic grounds common

The diode is reverse-biased while the coil is on. At turn-off, coil current continues through the diode rather than forcing a destructive voltage spike at the transistor. Place the suppression path near the coil/driver. A base-to-emitter resistor (for example 100 kΩ) can define the transistor's

off state if the buffer is disconnected. The 74HCT14's unused inputs need defined levels and its VCC/GND need bypassing.

**Worked example: Why a GPIO cannot directly energise this coil**

Nominal coil current is $5/167 = 29.9401$ mA and coil power is $25/167 = 149.7006$ mW. With an illustrative NPN saturation drop of 0.2 V, coil current is $4.8/167 = 28.7425$ mA. A forced gain of approximately 10 calls for about 2.9 mA base current. If the buffer's loaded output is about 4.0 V and $V_{BE} \simeq 0.8$ V, a 1 kΩ resistor gives about 3.2 mA. These are design estimates; check transistor saturation, buffer output-drive and relay pickup across actual supply/temperature conditions.

The nominal circuit avoids the weak MCU source driving the transistor base directly. The relay's guaranteed operate voltage must be below the voltage actually left across its coil, including driver drop. A Darlington array can replace the discrete stage, but its larger saturation drop must be included rather than automatically assumed harmless for a 5 V relay.

**Alternate relay on and off**

```
#include "timer_delay.h"
sbit RELAY_N = P1^0;
/* P1.0 -> 74HCT14 -> 1k base resistor -> 2N2222A.
   5V relay coil: +5V to collector; emitter=GND.
   1N4007 flyback diode: cathode=+5V, anode=collector. */
void main(void)
{
    IE = 0;
    RELAY_N = 1;            /* Off; also the normal reset-pin state. */
    while (1) {
        RELAY_N = 0;
        delay_ms(1000);     /* Coil energised, NO contact closes. */
        RELAY_N = 1;
        delay_ms(1000);     /* Coil released, NO contact opens. */
    }
}
```

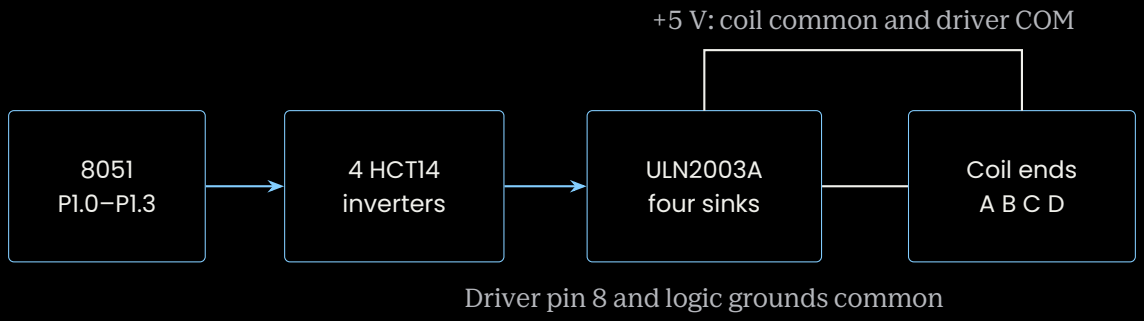
Here GPIO low means *coil on* because the HCT inverter outputs high. The relay changes approximately once per second; mechanical operate/release delays add to the electrical command. Contacts should switch only a load within the selected relay's ratings. A contact load may need its own suppression; the coil diode does not suppress an inductive contact load. Mechanical relays are unsuitable for fast PWM.

## 6.3 Stepper motors and excitation sequences

A **stepper motor** moves toward successive stable magnetic-field positions when its phases are energised in order. Open-loop positioning uses the commanded step count, assuming the load, speed and acceleration do not cause missed steps. Keeping the last excitation can provide holding torque; turning all phases off releases that torque. Direction follows sequence order; speed follows step interval.

The selected small demonstration motor is a **5 V 28BYJ-48**, a four-phase unipolar geared motor, driven by four **ULN2003A** channels through four HCT14 inverters. Its common winding lead goes to +5 V and each coil end goes to a driver collector. ULN2003A pin 8 is ground and pin 9 COM is

the common cathode of its internal clamp diodes: **connect COM to the positive winding supply**, not ground. Pin 1 input pairs with pin 16 output, pin 2 with 15, pin 3 with 14, and pin 4 with 13.

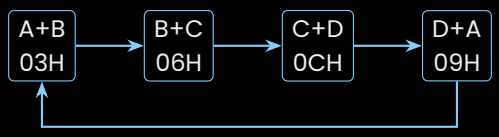



Coil order A–D is a *defined winding order*, not an assertion that wire colours from every supplier agree. Identify the common and phase order from the motor's sheet or resistance tests. A bipolar motor requires current reversal in each winding using H bridges; four simple low-side sinks are not a bipolar driver.

Let $Q_3Q_2Q_1Q_0 = D, C, B, A$, with 1 meaning an *energised coil*. These are driver-input patterns after the inverter, not the complemented port byte.

| State | Wave: one phase | Full step: two phases | Half step: alternating |
|---|---|---|---|
| 0 | 0001 (01H) | 0011 (03H) | 0001 (01H) |
| 1 | 0010 (02H) | 0110 (06H) | 0011 (03H) |
| 2 | 0100 (04H) | 1100 (0CH) | 0010 (02H) |
| 3 | 1000 (08H) | 1001 (09H) | 0110 (06H) |
| 4 | repeat | repeat | 0100 (04H) |
| 5 | | | 1100 (0CH) |
| 6 | | | 1000 (08H) |
| 7 | | | 1001 (09H) |

Wave drive uses one phase at a time. Two-phase full-step drive normally gives more torque with greater coil power. Half-step drive interleaves one- and two-phase states and halves the nominal angular increment; torque varies unless currents are adjusted. Reversing the row order reverses rotation. Repeating the same pattern holds position rather than making another step.



### Two-phase full-step drive, forward then reverse

```
#include "timer_delay.h"
/* 28BYJ-48 5V: four coil ends A..D through ULN2003A.
   P1.0..P1.3 -> four HCT14 inverters -> driver inputs.
   1 in logical table means ENERGISED, hence invert port bits. */
const unsigned char code seq[4] = {0x03,0x06,0x0C,0x09};
static unsigned char phase = 0;
static void move(unsigned int steps, bit reverse)
{
    while (steps != 0) {
        if (reverse) phase = (phase + 3) & 3;
        else phase = (phase + 1) & 3;
        P1 = 0xF0 | ((~seq[phase]) & 0x0F);
        delay_ms(10);       /* About 100 full-step changes per second. */
        --steps;
    }
    /* Hold last phase. To release coils deliberately, write P1=0xFF. */
}
void main(void)
{
    IE = 0;
    P1 = 0xFF;
    P1 = 0xF0 | ((~seq[phase]) & 0x0F);
    delay_ms(10);           /* Establish initial rotor alignment. */
    while (1) {
        move(200, 0);
        delay_ms(1000);
        move(200, 1);
        delay_ms(1000);
    }
}
```

The stored phase index persists across calls. Initial excitation establishes an alignment; each subsequent iteration advances or reverses exactly one sequence state. The output masks keep unused upper pins high, and inversion makes logical 1 energise its coil. Holding the final state maintains torque during the pause; write FFH to release the coils deliberately. The program makes 200 scheduled changes forward and then 200 backward; actual shaft return still depends on no lost steps/backlash. It does not claim that 200 changes equal one revolution of this geared motor.

**Worked example: Step angle, position and speed**

For an *assumed un-geared* motor with specified $1.8^\circ$ full-step angle, steps/revolution=$360/1.8 = 200$. A $90^\circ$ move needs $90/1.8 = 50$ full steps. At 500 full steps/s, speed=$60 \times 500/200 = 150$ rpm. With half steps, the nominal step angle is $0.9^\circ$, 400 changes/revolution, and 100 changes give $90^\circ$.

For the 28BYJ-48's published *nominal* $5.625^\circ/64$ stride convention interpreted as half-step operation, output increment=$0.087890625^\circ$, so 4096 half steps/revolution and 2048 two-phase full steps/revolution in that nominal model. Two-phase full-step increment=$0.17578125^\circ$; 200 full-step changes correspond to $35.15625^\circ$, and a 10 ms interval gives nominal $60 \times 100/2048 = 2.9296875$ rpm. Supplier gearing/step conventions

and backlash vary; use the actual motor's documentation or calibrated output revolution count for precision positioning.

The ULN2003A is a Darlington sink with a voltage drop, not an ideal short to ground; at nominal 5 V coil supply it reduces winding voltage/current. Respect coil and package power limits when holding two phases on. "500 mA per channel" is not permission to run every channel at that current simultaneously without a thermal check.

### 6.4 DC motor direction control using an H bridge

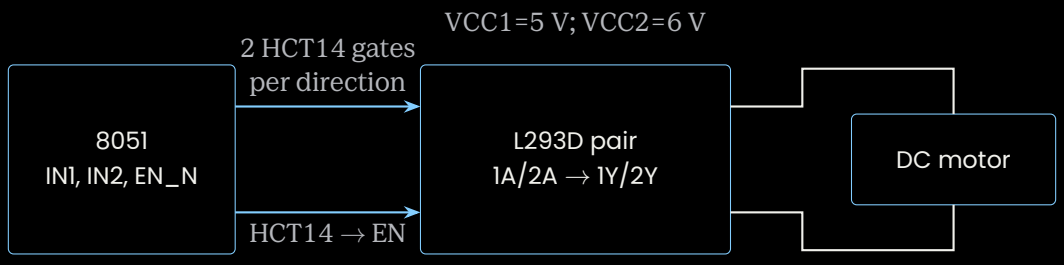
A brushed permanent-magnet **DC motor** rotates continuously when driven; reversing winding voltage reverses torque/direction. Unlike a stepper, it does not move a fixed angle for each input pulse. Position or accurate speed needs feedback. A low-side transistor controls one-direction on/off; an **H bridge** provides bidirectional voltage.

The selected driver is **L293D**, using its first two half bridges. Assume a **small 6 V brushed motor with verified stall current no greater than 400 mA**. This is the load specification for the example, not a claim about an unspecified physical motor. The driver's 600 mA/channel rating, voltage drop, transient current and thermal limits must all be checked against the motor. The un-suffixed L293 has different current/clamp provisions.

| L293D DIP connection | Function |
|---|---|
| 1A (2), 2A (7) | P1.0 IN1, P1.1 IN2, each through two cascaded HCT14 inverters (net non-inverting). |
| 1,2EN (1) | P1.2 *through* HCT14; software name EN_N. |
| 1Y (3), 2Y (6) | Motor terminals. |
| VCC1 (16), VCC2 (8) | +5 V logic, +6 V motor supply (example). |
| GND (4,5,12,13) | Common ground and heat-current return. |
| Unused driver side | Disable its enable and hold unused inputs at defined levels. |

The direction inputs are buffered too: the L293 input-current maximum is higher than the classic port's guaranteed high-source drive. Two HCT14 stages per direction preserve its logic polarity; one stage on EN supplies the required inversion. Five of the chip's six gates are used, with its spare input tied to a defined level. HCT outputs have ample drive for these logic inputs; motor current remains in the bridge.

Add close bypass capacitors to both supplies and a bulk capacitor on the motor rail. L293D integrates inductive clamp diodes; the external supply/return network must accept the recirculating energy. A diode placed simply across a reversible motor as if it were a relay coil would conduct during one commanded polarity and is not the correct H-bridge protection topology.




Grounds common; motor current stays in driver/supply loop

| Physical EN | IN1 | IN2 | Output / motor condition |
|---|---|---|---|
| 0 | X | X | Both outputs high impedance: coast. |
| 1 | 1 | 0 | 1Y high, 2Y low: one direction. |
| 1 | 0 | 1 | 1Y low, 2Y high: reverse direction. |
| 1 | 0 | 0 | Both low: dynamic braking. |
| 1 | 1 | 1 | Both high: dynamic braking. |

"Forward" depends on how the motor leads are connected. Equal bridge outputs provide zero winding voltage with a low-impedance path for braking, which differs from disabling outputs to coast.

**Run forward, coast, reverse and coast**

```
#include "timer_delay.h"
/* IN1/IN2 each use TWO HCT14 stages: polarity is preserved. */
sbit IN1  = P1^0;
sbit IN2  = P1^1;
sbit EN_N = P1^2;          /* HCT14 inversion: 0 enables L293D. */
void main(void)
{
    IE = 0;
    EN_N = 1;
    IN1 = 0;
    IN2 = 0;
    while (1) {
        IN1 = 1; IN2 = 0;
        EN_N = 0;
        delay_ms(2000);     /* Forward. */
        EN_N = 1;
        delay_ms(1000);     /* Coast before changing direction. */
        IN1 = 0; IN2 = 1;
        EN_N = 0;
        delay_ms(2000);     /* Reverse. */
        EN_N = 1;
        delay_ms(1000);
    }
}
```

Disable the bridge before changing direction; the one-second coast is a slow demonstration pause, not a guarantee that every loaded motor has stopped. Choose a suitable braking/coasting interval and verify current for the actual motor. HCT inversion makes EN_N=1 disable and EN_N=0 enable. PWM on that enable controls average drive while the direction inputs remain fixed.

**What the exam can ask; recurring mistakes**

Explain why a driver and flyback path are required, draw the relay transistor stage, list full/half-step sequences, calculate step angle/speed, or develop a DC H-bridge truth table and program. Errors: directly powering a coil from P1; omitting the buffer's current requirement; ULN COM tied to ground; confusing logical excitation with complemented GPIO; one revolution assumed from four table rows; reversing a spinning motor without a current/stop plan; coast called brake.

Verification and further reading: Relay ratings [RELAY]; transistor/diode selection [NPN], [DIODE]; HCT logic drive [HCT14]; ULN2003A clamps and thermal limits [ULN]; 28BYJ-48 nominal specification [STEPPER]; L293D truth table and supply limits [BRIDGE].

# 7 Pulse-width modulation using the 8051

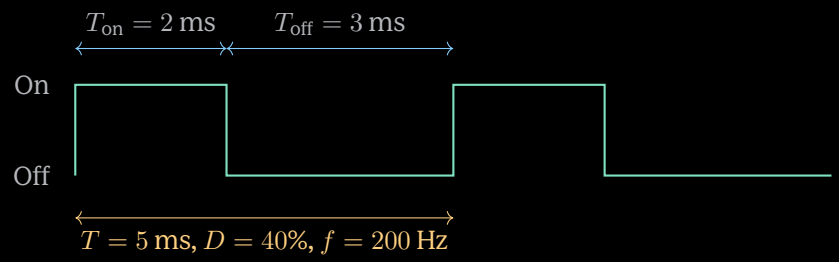
## 7.1 What PWM controls

**Pulse-width modulation (PWM)** varies a signal's on-time within a repeated period. A driver switches fully on/off while the motor's electrical/mechanical dynamics smooth its response. PWM is used for DC-motor speed control, LED brightness and, with filtering, an approximate analogue level. A relay's moving contacts are not suitable for this repeated fast switching.

For period $T$, on-time $T_{\text{on}}$ and off-time $T_{\text{off}}$:

$$T = T_{\text{on}} + T_{\text{off}}, \qquad f_{\text{PWM}} = \frac{1}{T}, \qquad D = \frac{T_{\text{on}}}{T}, \qquad D_{\%} = 100D.$$

An ideal unipolar switched waveform between 0 and $V_S$ has mean $DV_S$. This is a circuit average, not a guarantee that motor speed equals $D$ times rated speed. Driver drops, winding resistance/inductance, back EMF, load and current-recirculation mode affect actual speed and voltage. Use feedback for regulated speed.




The figure shows the *physical bridge enable*, high when on. The 8051 EN_N pin in the selected circuit has the inverse waveform because of the HCT14.

## 7.2 Classic software PWM versus derivative hardware PWM

A plain classic 8051 does not provide a dedicated hardware PWM peripheral. Software can toggle GPIO with timer-based waits, or a timer ISR can update outputs periodically. Modern 8051 derivatives may have capture/compare or PWM hardware; those features and registers must not be assumed for `reg51.h` and the classic architecture.

| Approach | Consequence |
|---|---|
| Blocking on/off waits | Easy demonstration; main spends the period waiting. Delay overhead changes frequency. Other enabled interrupts can still interrupt the wait and stretch edges. |
| Timer ISR with phase counter | Periodic hardware tick; foreground is available between ISRs. Output has ISR latency/jitter and an interrupt-time budget. |
| Hardware PWM on a derivative | Peripheral schedules edges with lower CPU cost, but uses that device's specific hardware/documentation. |

## 7.3 Timer 0 mode-2 PWM: a complete program

For this program only, use a **12 MHz, 12-clock core**. Timer 0 increments every 1 $\mu$s. Choose a 250-count auto-reload interval: TH0=TL0=$256 - 250 = 6 = 06$H. This gives a nominal tick rate of 4000

Hz. A 20-state phase counter gives a 5 ms PWM period, nominally 200 Hz, and 5-percentage-point duty steps.

The circuit is the L293D interface from section 6.4, with fixed forward inputs and PWM applied to its physical EN through the inverter. The PWM example owns Timer 0: do not include/call the polling `timer_delay.h` routine at the same time.

**200 Hz nominal PWM, initially 40% duty**

```
#include <reg51.h>
/* IN1/IN2 each use TWO HCT14 stages: polarity is preserved. */
sbit IN1  = P1^0;
sbit IN2  = P1^1;
sbit EN_N = P1^2;           /* HCT14 -> L293D EN; reset = off. */
/* THIS example uses a 12 MHz crystal on a 12-clock core.
   Timer 0 mode 2: 250 us tick; 20 ticks/period -> 200 Hz. */
volatile unsigned char duty_ticks = 8; /* 8/20 = 40%. */
static unsigned char phase = 0;
void timer0_isr(void) interrupt 1
{
    /* TF0 is cleared automatically on vectoring. */
    if (phase < duty_ticks) EN_N = 0;
    else EN_N = 1;
    ++phase;
    if (phase == 20) phase = 0;
}
void main(void)
{
    IE = 0;
    EN_N = 1;
    IN1 = 1; IN2 = 0;
    TMOD = (TMOD & 0xF0) | 0x02;
    TH0 = 0x06;
    TL0 = 0x06;
    TF0 = 0;
    ET0 = 1;
    EA = 1;
    TR0 = 1;
    while (1) {
        /* Set duty_ticks to a value 0..20 in foreground code.
           Latch updates at phase 0 if glitches must be avoided. */
    }
}
```

**How it works.** On each Timer 0 interrupt, compare phase (0–19) with duty_ticks (0–20). For phase smaller than duty_ticks, EN_N is low and the physical driver EN high. Otherwise the motor coasts with EN low. Increment and wrap phase at 20. TF0 is cleared by hardware on vectoring; no manual software reload is needed because mode 2 reloads TL0 from TH0.

**Endpoints.** duty_ticks=0 produces continuous off; 20 produces continuous on. At 8, eight on ticks and twelve off ticks give $8/20 = 40\%$. Values above 20 are outside the specified interface; validate inputs in a real user-control program. Duty is an 8-bit variable, so an individual byte update is indivisible on the classic core, but an update midway through a period can change the shape of that period.

To avoid changes within a period, use a requested duty and an ISR-local active duty; copy the requested byte to the active byte only when phase=0. This is a design refinement, not a need to disable interrupts for all duty changes. The following is a **C51 design fragment**, to replace the comparison logic inside the ISR after declaring the two variables:

```
/* Globals: volatile unsigned char requested = 8;
            static unsigned char active = 8; */
if (phase == 0) active = requested; /* requested constrained to 0..20 */
if (phase < active) EN_N = 0;
else EN_N = 1;
```

**Worked example: Frequency, width and mean voltage**

Timer tick: $(256-6)\times 1\,\mu\text{s} = 250\,\mu\text{s}$. PWM period: $20\times 250\,\mu\text{s} = 5$ ms; frequency=$1/0.005 = 200$ Hz. At 8 on ticks, $T_{\text{on}} = 2$ ms, $T_{\text{off}} = 3$ ms and duty=40%.
For an *ideal* 6 V unipolar switch, average voltage=$0.40\times 6 = 2.4$ V. If the selected bridge drops a total 2 V while conducting at the chosen current and its off-state motor-voltage model is approximately zero, a crude average is $0.40\times(6-2) = 1.6$ V. Actual bridge/freewheel voltage and speed require circuit/load measurements.
If the same reload is run at 11.0592 MHz, tick=$250/921600 = 271.267361\ \mu$s, period=5.425347222 ms and frequency=184.32 Hz. Reload alone does not specify frequency without a crystal/core assumption.

## 7.4 Timing limits, resolution and alternative calculations

Mode-2 timer overflow is periodic in hardware, but the GPIO is written only after interrupt entry/prologue. Masked interrupts or a higher/same-priority handler can postpone that write. Missed overflows are not stored in an unlimited queue; TF0 is a single bit. Keep worst-case ISR execution and blocking below the tick interval, and measure edge timing from generated code or on a scope before calling it exact.

If one ISR takes an *assumed* 50 machine cycles on a 12 MHz core, 4000 ISRs/s consume $4000\times 50/1000000 = 20\%$ of the CPU. This is a budgeting example, not a measured C51 cost. The program's compiled prologue/body/epilogue cost remains a native-toolchain verification item.

**Worked example: Resolution versus CPU cost**

With $M$ phase ticks, duty step is $1/M$ and $f_{\text{tick}} = M f_{\text{PWM}}$. A 20-tick scheme has 5% steps; a 100-tick scheme has 1% steps. A 256-tick, 1 kHz PWM would require 256000 ISR entries/s. On a 1 MHz machine-cycle rate that is only 3.90625 machine cycles per tick, far too little for a C ISR's entry/body/exit. Increasing resolution and frequency together quickly exceeds a classic 8051's software capacity.

For a *blocking 16-bit Timer 0* design at 12 MHz, exact hardware counts of 2000 and 3000 would yield reloads $65536-2000 = \text{F830}_{\text{H}}$ and $65536-3000 = \text{F448}_{\text{H}}$ for the 2 ms/3 ms intervals. GPIO writes and timer setup add time; do not present the combined software waveform as exactly 200 Hz without accounting for those instructions. At 11.0592 MHz the corresponding ideal counts are nonintegral and need rounding plus error calculation.

**Exam answer and common mistakes**

Define period/frequency/duty, show a waveform, state clock/core assumptions, derive timer counts and reload, show the output/driver path, and explain the ISR phase counter. Errors: assuming classic hardware PWM exists; duty=on/off rather than on/period; percentage treated as a raw tick count; using Timer 0 simultaneously for delays; changing direction inputs instead of enable for simple speed control; forgetting the interface inverts the GPIO waveform.

Verification and further reading: Classic timer/interrupt operation [INTEL], [NXP]; L293D enable and recirculation conditions [BRIDGE]; motor/PWM textbook chapter map [MAZIDI].

# 8 Answered exam practice and classroom exercises

**Use.** First answer from memory, then compare. Short answers are written for the syllabus's 3-mark style; the longer questions use 9 marks and at most three subdivisions. Suggested subdivisions are practice structures, not an official marking allocation. Diagrams, code and calculations cited below are part of the developed answers elsewhere in this guide and should be reconstructed when writing a full exam answer.

## 8.1 Three-mark questions: serial communication

**S1. Why choose serial instead of parallel for an 8051–PC link?**

**Model answer.** Serial uses fewer pins, cable conductors and connector contacts. It avoids skew among a parallel data bundle. The tradeoff is successive bit transfer and framing overhead; its speed still depends on the link clock and format.

**S2. Explain the frame and bit order of mode 1.**

**Model answer.** It is start (0), eight data bits D0 through D7, then stop (1): ten bit-times. The idle line is high and data is LSB first. 'A'=41H therefore sends data bits 1,0,0,0,0,0,1,0.

**S3. Why is a start bit needed without a clock wire?**

**Model answer.** The falling edge from idle high to start low aligns the receiver's bit timing for that frame. Classic mode 1 then samples near each bit centre. The stop interval restores the mark condition so another start can be recognised.

**S4. Distinguish UART and RS232.**

**Model answer.** The UART forms/decodes timed frames at MCU logic levels. RS232 specifies the external line interface and data polarity/voltages. MAX232 translates between these electrical domains but supplies no UART framing.

**S5. Give the RS232 data polarity and the invalid region.**

**Model answer.** Logic 1/mark is negative, below $-3$ V in the ordinary receiver-level description. Logic 0/space is positive, above $+3$ V. Between $-3$ and $+3$ V is undefined; a 0/5 V GPIO is not an RS232 driver.

**S6. State the MAX232 transmit and receive paths to a PC DTE.**

**Model answer.** P3.1 goes to T1IN, then T1OUT to PC DE-9 pin 2. PC pin 3 goes to R1IN, then R1OUT to P3.0. PC pin 5 and board signal ground are common; converter supply and charge-pump capacitors are required.

**S7. Why is SBUF not ordinary read-back storage?**

**Model answer.** Write and read access different physical registers at 99H: transmit and receive. A write requests sending; a read retrieves the accepted receive byte. This permits full duplex but cannot verify what was transmitted by reading SBUF.

**S8. Decode SCON=50H.**

**Model answer.** 50H=$01010000_2$. SM0=0, SM1=1 selects mode 1; SM2=0 gives ordinary reception; REN=1 enables RX. TB8, RB8, TI and RI begin clear.

**S9. A transmitter works but SCON=40H prevents receiving. Explain.**

**Model answer.** 40H still selects mode 1, so TX works. REN (bit 4) is 0; RX is disabled and arriving frames cannot set RI through normal reception. Set REN or initialise SCON=50H.

**S10. When is TI set, and does it prove reception?**

**Model answer.** In mode 1, hardware sets TI at the beginning of the stop bit, after the eight data bits. Software must clear it. It is a local transmit-ready event, not a remote acknowledgement or proof that the final stop interval has elapsed.

**S11. Who clears TI and RI, and what goes wrong if they stay set?**

**Model answer.** Software clears both. A stale TI can make a send wait exit too soon. A stale RI makes old data appear pending and prevents acceptance of a subsequent frame. With ES/EA enabled, either uncleared flag also keeps serial service requested.

**S12. Why normally use Timer 1 mode 2 for the UART?**

**Model answer.** TL1 automatically reloads from TH1 on overflow, giving repeated periodic baud-clock overflows without software reload. TR1 must start it and ET1 normally stays clear. Other timer configurations need separate timing justification.

**S13. Derive TH1 for 9600 baud at 11.0592 MHz, SMOD=0.**

**Model answer.** $N = f_{\text{osc}}/(384B) = 3$ counts; TH1=$256 - 3 = 253$ =FDH. Load TL1=FDH as well, select Timer 1 mode 2 and start TR1. The nominal baud error is zero.

**S14. Why 11.0592 MHz rather than 12 MHz for this UART configuration?**

**Model answer.** At TH1=FDH and SMOD=0, 11.0592 MHz gives 9600 exactly; 12 MHz gives 10416.6667 baud, $+8.506944\%$ error. The crystal is chosen for convenient integer division, not because 12 MHz is intrinsically forbidden.

**S15. What if TR1=0 after all UART registers have been set?**

**Model answer.** Timer 1 does not overflow, so the selected UART baud generator has no progressing clock. A normal transmission cannot complete and a polling TI loop waits. Configuration and starting are separate actions.

**S16. What is the main cost of a blocking RI/TI polling loop?**

**Model answer.** It occupies foreground execution while waiting, leaving no time for other main-loop work. RI waiting has no upper bound if nobody sends. A TI-ready wait is roughly nine bit-times plus initial alignment in mode 1, while complete frame wire time is ten bit-times. Enabled interrupts can still run.

## 8.2 Three-mark questions: interrupts and interfaces

**S17. List the classic 8051 interrupt sources.**

**Model answer.** INT0, Timer 0 overflow, INT1, Timer 1 overflow and the serial port (RI OR TI). These are five vectors. Reset is distinct; an 8052 adds the Timer 2 vector.

**S18. Which IE value enables only serial service?**

**Model answer.** 90H sets EA (bit 7) and ES (bit 4), leaving the other classic source enables clear. 10H leaves global service disabled. IP determines priority separately.

**S19. State the serial vector and Keil ISR declaration.**

**Model answer.** Vector 0023H, C51 number 4: `void serial_isr(void) interrupt 4`. The function takes no arguments and returns no value; the compiler supplies appropriate entry/exit code and RETI.

**S20. Configure external interrupt 1 for a falling edge.**

**Model answer.** Release P3.3's latch, set IT1=1, clear stale IE1, set EX1=1 and EA=1. Declare the handler with `interrupt 2`, vector 0013H. A raw button still needs debounce.

**S21. Why does ES=1 alone not guarantee an ISR?**

**Model answer.** EA must also be 1 and RI or TI must request service. Correct vector, priority eligibility and the serial hardware configuration are needed. Setting an enable does not create a received character.

**S22. Why test both RI and TI in the serial ISR?**

**Model answer.** They share one vector. Either or both can be set. Independently test each, read SBUF before clearing RI, and acknowledge TI without assuming a transmit event is a receive event.

**S23. What if the ISR never clears RI?**

**Model answer.** Serial service remains pending and can recur after eligible return points; the foreground is repeatedly interrupted. The unread holding register also prevents a later mode-1 frame transfer. Capture and clear RI in software.

**S24. Why can holding a button low give repeated interrupts?**

**Model answer.** IT0=0 selects low-level requests, so the condition persists after return. IT0=1 instead latches falling edges. Multiple edges from mechanical bounce can still occur and must be filtered.

**S25. How many priority levels exist, and what does IP choose?**

**Model answer.** Two: high and low. Each IP source bit chooses its level, with 1 high. Within a level, simultaneous eligible pending requests use a fixed polling order; that order does not create extra nesting levels.

**S26. At IP=00H, INT0 and serial pending together: which first?**

**Model answer.** Both are low. INT0 is selected first by the fixed order INT0, T0, INT1, T1, serial. A still-pending serial request is served later when eligible.

**S27. Can low INT0 pre-empt an active high serial ISR?**

**Model answer.** No. It waits if its request remains pending. Only a high request can pre-empt an active low ISR; equal priorities do not pre-empt even if one source has a better tie rank.

**S28. Why volatile, and why does it not solve all sharing?**

**Model answer.** Volatile prevents inappropriate optimisation of asynchronously shared accesses. It does not make a two-byte read or a test-copy-clear sequence atomic. Use a short critical section for a consistent transfer and a queue/counter where several events must be retained.

**S29. What distinguishes an LCD command from an ASCII data byte?**

**Model answer.** RS=0 for commands and RS=1 for character data. Thus 38H with RS=0 configures the display while 38H with RS=1 writes '8'. E must be strobed and the instruction completion time respected.

**S30. Why does LCD clear need a special wait?**

**Model answer.** Clear/home take about 1.52 ms at the reference controller clock, while most commands take about 37 $\mu$s. Time scales with its clock. A short ordinary-character delay is not enough; wait conservatively or read busy after initialisation.

**S31. Define ADC resolution for an 8-bit 0–5 V span.**

**Model answer.** There are 256 codes. One ideal LSB is $5/256 = 19.53125$ mV. Resolution is quantisation spacing, not proof of that accuracy; converter/reference/noise errors also matter.

**S32. State the ADC0804 start and read handshake.**

**Model answer.** CS low; pulse WR low then high to start. Wait until INTR is low, assert RD low, wait access time and read the released data port. Return RD high; INTR and RI are different signals.

**S33. Give the DAC0808 output for D=128 and 2 mA reference.**

**Model answer.** $I_{OUT} = 2 \times 128/256 = 1$ mA. The selected current-sink I/V stage with 2.5 kΩ feedback gives +2.5 V. State the circuit/current direction before choosing output sign.

**S34. Why use a flyback diode on a relay coil?**

**Model answer.** Inductive current cannot stop instantly at turn-off. The diode provides a recirculation path and limits the transistor voltage. Cathode goes to the positive coil supply, anode to the switched low side; it is reverse-biased during normal energisation.

**S35. Distinguish full and half stepping.**

**Model answer.** Two-phase full stepping repeats 03H, 06H, 0CH, 09H in the defined coil order. Half stepping interleaves one-phase states and gives half the nominal increment. Reverse sequence order reverses direction; driver polarity may require complementing GPIO bits.

**S36. Distinguish coast and brake on the L293D interface.**

**Model answer.** EN=0 makes outputs high impedance and lets the motor coast. With EN=1 and equal direction inputs, both outputs have the same driven level and provide dynamic braking. Opposite inputs drive the two directions.

**S37. Calculate PWM for 2 ms on and 3 ms off.**

**Model answer.** $T = 5$ ms, $f = 200$ Hz, duty=$2/5 = 40\%$. An ideal 6 V switch gives a 2.4 V mean. Actual motor behaviour depends on driver, freewheel path and load.

**S38. Does a classic 8051 have a general-purpose hardware PWM peripheral?**

**Model answer.** No. Generate PWM with software and timers/ISRs, or select a derivative with documented PWM hardware. A mode-2 tick counter trades duty resolution and frequency against CPU/interrupt cost.

## 8.3 Nine-mark full questions with developed answers

### L1. Serial interface and an independent baud calculation

**Question.** (a) Draw a classic 8051–PC RS232 interface and explain its purpose. (b) Configure 4800-baud mode 1 using 11.0592 MHz and SMOD=0. (c) Explain an 'A' transmission, its frame time and flags.

**Developed answer.** (a) Draw P3.1 to MAX232 T1IN/T1OUT to PC DTE receive pin 2, and PC transmit pin 3 through R1IN/R1OUT to P3.0. Join signal grounds to PC pin 5. Add +5 V, charge-pump capacitors and bypassing. The UART supplies framing; the MAX232 supplies voltage/polarity translation between 0/5 V logic and the bipolar inverted RS232 data interface.

(b) $N = 11059200/(384 \times 4800) = 6$, so TH1=FAH and TL1=FAH. Stop Timer 1, preserve Timer 0 using `TMOD=(TMOD&0x0F)|0x20`, clear SMOD with `PCON&=0x7F`, set SCON=50H, keep ET1=0 and start TR1. Nominal baud is exactly 4800. The complete initialisation/header and send function in section 2.5 apply with FDH changed to FAH in both reload registers.

(c) Clear stale TI, write 41H to SBUF, wait for TI and clear it. Draw start 0; data $1, 0, 0, 0, 0, 0, 1, 0$; stop 1. Each bit takes 208.333333 $\mu$s and wire frame time is 2.083333333 ms. TI is set at the start of stop, about nine bit-times after the frame start, not after the full stop. SBUF read-back is the receive holding register and TI provides no acknowledgement.

#### L2. External interrupt, priority and a shared counter

**Question.** (a) Explain edge/level INT0 operation. (b) Explain simultaneous INT0/serial and nested service for IP=10H. (c) Show how C51 main safely reads a 16-bit ISR-updated count.

**Developed answer.** (a) P3.2 must be an input with released latch. With IT0=1, a detected high-to-low transition sets IE0, which clears on vectoring; holding low creates no further edge by itself. With IT0=0, a held low keeps requesting service; the source must release it. Both require EX0 and EA. A switch additionally needs debounce; edge selection alone cannot suppress contact transitions.

(b) IP=10H means PS=1 and PX0=0. Simultaneous eligible requests select serial first because high outranks low. An INT0 request during an active high serial ISR waits. Conversely a high serial request can suspend a running low INT0 ISR, execute, then return to INT0 before main. Fixed vector order applies only among eligible requests at the same level.

(c) Declare `volatile unsigned int count`. Increment in `void int0_isr(void) interrupt 0`. In main save EA, clear EA, copy count to a local 16-bit snapshot, then restore saved EA before display/formatting. This prevents mixed old/new bytes. Hardware saves PC; the compiler protects used context and emits RETI. Use the complete clean-pulse program on page 17; a raw button needs the following debounce example.

#### L3. LCD output, addressing and timing

**Question.** (a) Draw the selected 8-bit LCD interface. (b) Explain initialisation and output at second-line column 5. (c) Explain two likely causes of a blank/incorrect display.

**Developed answer.** (a) Use a 16×2 HD44780-compatible module: VDD=+5 V, VSS=ground, V0=contrast potentiometer wiper, D0–D7=P2, RS=P3.4, E=P3.5 and R/W=ground for writes. Backlight uses its own rated supply/current limiting. Internal controller memory stores character codes.

(b) Wait after valid power; perform the three 30H recovery writes with the documented early waits; send 38H for 8-bit/two-line/5×8 operation, 08H display off, 01H clear, 06H increment/no shift, 0CH display on. Row 2 starts at DDRAM 40H; fifth column counting from 1 adds 4, so send C4H with RS=0. Send character bytes with RS=1, valid data setup, an E high/low pulse and completion waits. The full LCD program gives an executable structure, including its delay header.

(c) A powered but blank module may have incorrect contrast or no successful initialisation. Incorrect characters can result from swapped data wires, malformed enable pulses or insufficient completion waits. Clear/home take far longer than ordinary data writes. In four-bit mode, both nibbles must be sent in the correct order; merely halving the bus wires does not convert this program.

#### L4. ADC and DAC, with numerical interpretation

**Question.** (a) Explain ADC0804 GPIO reading. (b) Find the resolution and nominal voltage for code 180 with VREF/2=1.28 V. (c) Find DAC0808 output current and voltage for code 192 with the guide's reference/I-V circuit.

**Developed answer.** (a) CS is low, VIN− grounded, VIN+ the signal and reference/clock provided. Set the MCU data-port latch to FFH. Pulse WR low then high, wait for INTR low, assert RD low, wait access time, copy DB0–DB7 and release RD. Provide timeout/error behaviour rather than waiting forever if the ADC clock is absent. The 10 kΩ/150 pF oscillator is nominally 606.0606 kHz; conversion timing is external to the CPU.

(b) Span=$2 \times 1.28 = 2.56$ V; LSB=$2.56/256 = 0.01$ V. Code 180=B4H has centre estimate $180 \times 0.01 = 1.80$ V. State the centre convention and recognise ideal half-LSB quantisation plus real converter/reference error; do not use 255 as the denominator here.

(c) $I_{\text{REF}} = 5/2500 = 2$ mA and $D = 192$ =C0H. $I_{\text{OUT}} = 2 \times 192/256 = 1.5$ mA. With the selected current-sink output into a virtual-ground node and $R_f = 2.5\,\text{k}\Omega$, $V_{\text{out}} = +3.75$ V. A DAC0808 needs reference, supplies, compensation and an I/V stage; it is not directly a 3.75 V logic-pin output.

#### L5. Relay and stepper drive

**Question.** (a) Draw a protected relay interface. (b) Explain two-phase full-step operation and reversal. (c) For an assumed un-geared 1.8° motor, find changes for 45° and speed at 400 changes/s.

**Developed answer.** (a) Draw the inverter/buffer, base resistor and low-side NPN, with coil from +5 V to collector and emitter to ground. Draw the diode cathode at +5 V, anode at collector. GPIO controls base current through the buffer; the load supply powers the coil. Contacts COM/NO/NC form a separate switched-load path. In this circuit GPIO low means on because the HCT14 inverts it. Observe coil pickup and contact ratings.

(b) Define coil order A,B,C,D and logical 1 as energised. Apply A+B, B+C, C+D, D+A: 03H, 06H, 0CH, 09H. Each transition advances a full step; reverse the sequence to reverse motion. With ULN2003A sinks, winding common and clamp COM both go to the positive winding rail. The HCT14 design complements the lower GPIO nibble. Holding a state maintains torque; all off releases it. This is unipolar control; a bipolar winding needs an H bridge.

(c) Full steps/revolution=$360/1.8 = 200$. A 45° move needs $45/1.8 = 25$ changes. Speed=$60 \times 400/200 = 120$ rpm, assuming no missed steps. Half stepping changes the relevant angle and count. These un-geared numericals do not apply directly to the geared 28BYJ-48 demonstration.

#### L6. DC direction control and timer PWM

**Question.** (a) Explain L293D direction/coast/brake states. (b) Design 200 Hz, 40% nominal PWM with a 12 MHz classic 8051. (c) State practical timing/driver constraints.

**Developed answer.** (a) With EN high, inputs 10 and 01 give opposite output polarities; 00 or 11 gives equal outputs for braking. EN low gives high-impedance outputs and coast. VCC1 is 5 V logic, VCC2 the motor rail, outputs connect to the two motor terminals, and grounds are common. The selected inverter makes software EN_N opposite to physical EN.

(b) One machine cycle is 1 $\mu$s. Timer 0 mode 2 with TH0=TL0=06H counts 250 increments, so the tick is 250 $\mu$s. Count 20 ticks per period and turn on for 8: $T = 5$ ms, $f = 200$ Hz, on=2 ms, off=3 ms and duty=40%. Enable ET0 and EA, start TR0 and use `interrupt 1`. The ISR compares phase<8, sets EN_N low/high accordingly, increments phase and wraps at 20. TF0 clears on vectoring. The full program is in section 7.

(c) Timer 0 cannot also be overwritten by a delay function. ISR execution/latency must fit the 250 $\mu$s interval; mode 2 fixes the timer tick but not exact software edge timing. Check stall current, bridge drop and dissipation, supply decoupling and clamp paths. An ideal 6 V waveform

averages 2.4 V; real motor speed is not an exact 40% scaling. Native compilation and physical measurements are needed for a verified frequency guarantee.

#### L7. A receive interrupt does not prove loss-free operation

**Question.** (a) Analyse a 9600-baud receiver with 100 ms foreground work and one-byte mailbox. (b) Explain a suitable buffer/policy. (c) Explain the TI polling conflict and a remedy.

**Developed answer.** (a) Continuous 8N1 input supplies 960 bytes/s, about 96 bytes in 100 ms. A one-byte mailbox cannot retain that burst even if every ISR captures hardware SBUF promptly. If interrupts are masked for 100 ms, hardware loss occurs earlier: RI prevents a later frame transfer while set. Volatile alone solves neither capacity nor atomic handoff.

(b) Use a ring with consistent publication/removal, explicit full detection and an overflow indicator; choose capacity and service rate for a bounded workload. The guide's 16-slot/15-byte ring alone covers only 15.625 ms at the ideal arrival rate. A 100 ms burst needs capacity for roughly 96 bytes plus boundary/service margin, which is expensive on 128-byte classic RAM; shorten/decompose the task or use protocol flow control. Do not claim all inputs can be buffered indefinitely. Preserve the old queued data and reject/count new data on full, or deliberately choose another documented policy.

(c) An ISR clearing TI can remove the event main's hardware-flag wait is watching, leading to a hang. Keep polling serial with ES off, or let the ISR exclusively clear TI and set a volatile software-ready notification. The command program uses the latter. Formatting and replies stay out of the ISR; the TX queue demonstrates a bounded nonblocking alternative.

### 8.4 Code reading and debugging: the eight classroom faults

The classroom examples are diagnostic fragments with assumed helpers. Here each fault is answered with its reason and a complete-program destination. Several real fragments can have more than the single advertised defect; inspect flag ownership, sharing and hardware as well as the highlighted line.

| | Symptom / fragment | Diagnosis and correction |
|---|---|---|
| 1 | TX wait never completes; no `TR1=1`. | Configured Timer 1 is stopped. Start it after loading TH1/TL1 and setting SCON. Check actual clock/wiring too. Complete code: `serial_tx.c`. |
| 2 | Second send returns immediately; TI never cleared. | TI remains from the prior event. Clear stale TI before starting and acknowledge afterwards, with exclusive polling ownership. Complete header: `uart_poll.h`. |
| 3 | TX works, RX never sets RI; `SCON=0x40`. | REN=0. Select 50H or set REN. Complete code: `serial_rx.c`. |
| 4 | `ES=1`, ISR never runs; EA not set. | Set EA after initialisation. Also verify vector number and an actual pending flag. Complete code: `serial_irq_command.c`. |
| 5 | ISR reads SBUF unconditionally and clears no flags. | Test RI before reading, clear it after capture, and independently acknowledge TI. Otherwise TX events are mistaken for RX and serial service repeats. |

| | Symptom / fragment | Diagnosis and correction |
|---|---|---|
| 6 | Held button repeats with `IT0=0`. | Use edge mode for the press event, and add debounce. IT0=1 alone does not suppress bounce. Complete code: `external_button.c`. |
| 7 | Main misses changes after optimisation; nonvolatile shared globals. | Declare shared objects volatile. Also fix the test/copy/clear race with a short critical section or queue; volatile alone is not the whole correction. |
| 8 | Serial ISR parses/transmits a long reply. | Capture/enqueue and exit; process/reply in main. Check reentrancy and flag ownership. Prompt ISR service still needs bounded masks/priority and queue capacity. |

**D9. What is wrong with `receive: RI=0; return SBUF;`?**

**Model answer.** It opens the holding register to acceptance of a later completed frame before the old byte is copied. Prefer local=SBUF, then RI=0, then return local. The order is significant despite both lines being individually legal.

**D10. Why can `if(RI)... else if(TI)...` cause an extra ISR entry?**

**Model answer.** If both are set, only RI is handled in that entry. TI remains active. Two independent tests service both causes; neither flag is automatically cleared on entry.

**D11. Predict `ch=SBUF` immediately after `SBUF='X'`.**

**Model answer.** It reads the receive register's current value, which need not be 'X' and need not represent a new character without RI. There is no transmit read-back guarantee.

**D12. Decode IP=05H and IE=85H.**

**Model answer.** IP makes INT0 and INT1 high (PX0, PX1). IE enables EA, EX0 and EX1. Simultaneous eligible external requests both high select INT0 by tie order, but INT1 cannot pre-empt an already active high INT0 ISR.

**D13. Why does an ADC read as zero if P1=00H?**

**Model answer.** The MCU drives the shared data bus low; it is not released for the ADC's output. Set P1=FFH before plain port reads and ensure RD/CS enable the ADC only for the read.

**D14. LCD line 2 code sends `0x90`. What is wrong?**

**Model answer.** 90H sets DDRAM address 10H, not 40H. A common $16\times2$ module's second visible line starts at 40H, so use C0H for column 1.

**D15. A motor runs when GPIO=0, despite an "active-high" driver. Why?**

**Model answer.** The selected HCT14 interface inverts the MCU control. GPIO low produces high driver input/enable. Interpret the actual hardware chain and use names such as EN_N; do not infer load polarity from a library label.

**D16. What does PWM phase=7 with duty_ticks=8 produce?**

**Model answer.** Since $7 < 8$, EN_N=0 and physical EN=1. At phase=8 the enable turns off. Eight of twenty phase values are on; setting duty=20 gives continuous physical enable.

## 8.5 Classroom lab traces, viva answers and extensions

| Lab | Static expected behaviour and answered extension |
|---|---|
| 1. Repeated TX | Repeated ‘A’ frames draw START, 1 0 0 0 0 0 1 0, STOP. With no extra gaps, ideal maximum is 960/s at 9600. A 40-character extension costs 41.6667 ms wire time; use the TX program’s string loop, repeating the outer loop if required. |
| 2. RX display | REN must be 1. A received ‘A’ gives code 41H; active-low display port=BEH. A received-byte count increments once per captured frame, but cannot reveal hardware-discarded frames without a protocol sequence number. |
| 3. Echo | Capture RX, release RI, transmit captured byte. In an isolated exchange total wire time is about 2.0833 ms at 9600. Convert lowercase to uppercase by subtracting 20H in the ASCII lowercase range. RX/TX can overlap. |
| 4. Polling cost | A foreground LED toggled once per received byte stops toggling while a blocking RI wait has no input. This does not prove the CPU crashed. An ISR-driven queue allows short foreground work between events; do not interpret an invisible rapid toggle as a measured useful-work percentage. |
| 5. External event | Clean falling edge selects vector 0003H and C51 number 0; IE0 clears on entry. For a real switch, use the debounce program. A long-held low in level mode repeatedly requests service. |
| 6. Serial event | RI requests 0023H; ISR copies SBUF, clears RI and publishes data. TI uses the same vector. A queue replaces the unsafe single shared byte when several commands can arrive. |
| 7. Priority | IP=01H lets INT0 pre-empt a low serial ISR. A deliberately long ISR can demonstrate nesting, but is not good production design. If a nested handler writes shared output state, define ownership rather than assuming “last” remains unchanged. |
| 8. Commands | Queue ‘1’, ‘2’, ‘S’, ‘0’: replies ‘1’, ‘3’, ‘3’, ‘0’; LED states follow that sequence. Unknown command replies ‘?’. This corrected implementation uses ISR-owned TI, short capture and finite RX capacity. |

## 8.6 The classroom graded practice, answered and mapped

The five beginner tasks are answered by S7/S10 (SBUF and TI), S8 (50H), L1(c) (write/send), S9/S15 (receive wait causes), and S19 (serial vector). The five intermediate tasks are answered by L1(b) (4800 baud), the serial ISR/queue in section 3.8, S20 (INT1 setup), the priority rules in section 3.5 (same-level INT1 precedes serial), and L7 (why long replies do not belong in an ISR).

The five advanced tasks are answered by the corrected command program (capture, process and reply), the IP=01H nesting case (urgent external input), the bounded TX ring program (atomic kick and full policy), L7’s loss-budget analysis (measures required for no loss under a stated workload), and section 2.4’s independent 11.0592/12 MHz calculations. Undefined classroom helpers such as `display`, `handle` and `send_str` are teaching placeholders, not unresolved dependencies in the supplied standalone programs.

# 9 Final revision sheet and completion checklist

### Serial: rebuild this from memory

Mode 1 is **8N1**: start 0, D0–D7 LSB first, stop 1; logic-level idle high. UART frames bytes; MAX232 translates logic/RS232 levels. PC DTE pin 2 receives, pin 3 transmits, pin 5 is signal ground. TTL adapters and RS232 adapters are different interfaces.
SBUF=99H: **write TX, read RX**. SCON=98H: SM0 SM1 SM2 REN TB8 RB8 TI RI. 50H means mode 1, RX enabled. TI is set **at the beginning of stop**; RI near the accepted stop sample. Software clears both. Read SBUF *before* clearing RI. RI left set causes the *new frame* to be discarded.
$B = 2^{\mathrm{SMOD}} f_{\mathrm{osc}}/[384(256 - \mathrm{TH1})]$ for Timer 1 mode 2 on a 12-clock core. At 11.0592 MHz, SMOD=0: 1200/E8H, 2400/F4H, 4800/FAH, 9600/FDH. For 19200 use SMOD=1, FDH. TH1 and TL1 both loaded, TR1=1, ET1=0. Preserve Timer 0 with a masked TMOD write. PCON is not bit-addressable.
At 9600: 104.1667 $\mu$s/bit, 1.041667 ms/8N1 frame, ideal maximum 960 characters/s. A nominal 0% divider error does not remove oscillator tolerance or software deadlines.

### Interrupts: cause, enable, priority, vector, acknowledgement

IE=A8H: EA – – ES ET1 EX1 ET0 EX0. IP=B8H: – – – PS PT1 PX1 PT0 PX0. TCON=88H: TF1 TR1 TF0 TR0 IE1 IT1 IE0 IT0. Reserved bits clear on the classic 8051.
INT0/0003H/0; T0/000BH/1; INT1/0013H/2; T1/001BH/3; Serial/0023H/4. Timer 2/002BH/5 is an **8052 addition**. Reset=0000H is separate.
Two levels. High pre-empts low; equal cannot pre-empt; active high cannot be pre-empted by another classic maskable source. Same-level tie order: INT0, T0, INT1, T1, serial. ITx=1 falling edge; ITx=0 held-low request. Edge flags/timer flags clear on vectoring; RI/TI do not. RETI releases the in-service condition.
Keil: `void name(void) interrupt n. using n` selects a register bank, not priority. Short ISRs; capture then process in main. Volatile is not atomicity. Protect multibyte snapshots and copy/clear handoffs. Queues need full/empty rules and overflow policy. A serial ISR clearing TI conflicts with foreground hardware-TI polling; use one owner and a software-ready notification.

### Interfaces and motor control

LCD: RS=0 command, RS=1 data, R/W=0 write, pulse E. 38H function set, 06H increment, 0CH display on, 01H clear; line starts 80H/C0H. Clear/home need longer completion time. In four-bit mode send high then low nibble.
ADC0804: CS low, WR low/high starts, INTR low completes, RD low enables data; release MCU bus to FFH before reading. Span=$2V_{\mathrm{REF}/2}$; LSB=span/256. 5 V span gives 19.53125 mV/LSB. $V \approx Dq$ uses the stated code-centre convention.
DAC0808: current output, $I = I_{\mathrm{REF}} D/256$; FFH is 255/256 of the endpoint. Reference, negative supply, compliance and I/V stage matter. The guide's sink-current I/V stage gives positive $IR_f$.
Relay: power driver, coil flyback diode, separate contacts. Stepper logical full-step pattern 03H, 06H, 0CH, 09H; half-step 01H,03H,02H,06H,04H,0CH,08H,09H. Reverse order for reverse direction. ULN COM to winding supply; bit patterns depend on inverter wiring.
DC bridge: enabled 10/01 drive opposite directions, 00/11 brake; disabled coasts. Check stall current and driver drop. PWM: $D = T_{\mathrm{on}}/T$, $f = 1/T$; ideal mean=$DV_S$. At 12 MHz, Timer 0 mode 2 reload 06H gives 250 $\mu$s ticks; 20 ticks=200 Hz nominal; 8 on ticks=40%. Classic 8051 has no general hardware PWM peripheral.

## 9.1 Syllabus completion checklist

Check a box after you can answer from memory and reproduce its essential diagram, code and calculation.

| | Skill to demonstrate | Guide location |
|---|---|---|
| ☐ | Define serial/parallel, synchronous/asynchronous and duplex forms; draw an 8N1 frame. | 2.1, p. 8 |
| ☐ | Draw both MAX232 signal paths and explain logic/RS232 levels. | 2.2, p. 9 |
| ☐ | Decode SCON/PCON, calculate baud/reload/error, write TX/RX/echo code. | 2.5, p. 12 |
| ☐ | Explain five vectors, IE/IP/TCON, entry/return and flag clearing. | 3, p. 15 |
| ☐ | Configure both external interrupts and explain edge/level/bounce. | 3.4, p. 17 |
| ☐ | Handle RI and TI, protect sharing, explain loss and queue full policy. | 3.7, p. 22 |
| ☐ | Predict simultaneous and nested requests; distinguish tie order from levels. | 3.5, p. 20 |
| ☐ | Write and explain C51 ISR declarations and safe foreground handoff. | 3.6, p. 21 |
| ☐ | Draw LCD connections, initialise and write both lines with valid timing. | 4, p. 27 |
| ☐ | Draw ADC0804 handshake, read its bus and calculate code/voltage/resolution. | 5.2, p. 31 |
| ☐ | Explain DAC0808 current/reference/I-V circuit and output calculation. | 5.3, p. 33 |
| ☐ | Draw a relay driver and flyback path; distinguish coil and contact ratings. | 6.2, p. 35 |
| ☐ | Reproduce stepper wave/full/half sequences; calculate step angle and speed. | 6.3, p. 36 |
| ☐ | Draw L293D connections and explain drive/coast/brake code. | 6.4, p. 39 |
| ☐ | Derive PWM frequency/duty/reloads and explain ISR timing limitations. | 7, p. 41 |

**Before writing an exam program:** name core, clock and compiler; define hardware polarity; state peripheral ownership; initialise before enabling; match vector/number; acknowledge the right flags; and justify all numerical constants. For a long interface answer, include the load supply/driver and completion timing, not just a sequence of register assignments.

## 10 Discrepancy appendix: corrections and simplifications

This appendix records material differences from the supplied classroom HTML, including its JavaScript-generated content. It does not treat an illustrative omitted helper as a hardware error. The classroom chain, flag emphasis and short-ISR pattern remain useful; the corrections below are necessary when the material becomes an independent exam/reference guide.

| HTML location | Finding / classification | Resolution and evidence |
|---|---|---|
| SBUF/SCON slide, TX steps and quiz 10 | **Actual timing error:** TI described at end of stop bit. | Classic modes 1–3 set TI at beginning of stop; RI is halfway through the accepted stop interval. Intel SCON Figure 14, printed p. 3-15 (physical PDF p. 123), and mode-1 timing; [INTEL], [NXP]. |
| Receive slide, terminal simulator, several lab/bug explanations | **Actual hardware-model error:** a new frame said to overwrite an unread old byte while RI=1. | RI=0 is a condition for transfer to holding SBUF. With RI=1, the *new completed frame* is lost and unread old byte remains. Intel “More About Mode 1”, printed p. 3-20 (physical p. 128); [INTEL]. |
| Receive helper and polling labs | **Unsafe ordering:** RI cleared before SBUF read. | Copy SBUF first, then clear RI. This closes the window for a new accepted frame before old-byte capture. Derived from the documented acceptance rule; [INTEL], [NXP]. |
| Baud section/quiz asks why mode 2 “must” be used | **Overstatement of normal design choice.** | Mode 2 is the preferred automatic periodic clock. Intel explicitly permits other Timer 1 running modes and gives software-reload low-rate examples, printed p. 3-16 (physical p. 124); [INTEL]. |
| Polling/interrupt lab | **Overstatement:** interrupt-driven receiver said to have no service constraint. | Both designs have hardware capture deadlines; ISR latency, interrupt masks and finite queues still matter. The slot-usage display is illustrative, not a measured CPU benchmark. See section 3.8; [INTEL]. |

| HTML location | Finding / classification | Resolution and evidence |
|---|---|---|
| Integrated / advanced command example | **Incomplete safety:** shared single-byte command and flag cleared after handling; ISR also clears TI while reply helpers are undefined. | Test-copy-clear atomically or use a bounded queue. If reply helpers poll TI, an enabled TI-clearing ISR can deadlock them. Actual helper implementation is absent, so the TI conflict is a *conditional design flaw*, not a claim that an unseen helper must be broken. Corrected complete program owns TI in the ISR. |
| Advanced TX ring | **Incomplete implementation:** unbounded head/tail increments with masked indexing, no full test and unprotected idle kick. | Use bounded modulo indices, explicit capacity/full policy and atomic publish/kick. Otherwise unread bytes can be overwritten and a byte stranded during ISR/foreground interleaving. The corrected TX queue supplies these invariants. |
| Counter/ISR examples | **Teaching simplification:** volatile 16-bit count displayed without a protected snapshot. | Volatile does not make two-byte access atomic. Copy under a short saved-EA critical section. C51 type/interrupt rules [KEILTYPE], [KEILIRQ]. |
| External lab and bug 6 | **Overstatement:** edge mode implies exactly one count for a physical button press. | One clean falling edge gives one pending event. Contact bounce can create several; the HTML itself also mentions bounce. A debounce circuit/state sequence is needed for one physical press. |
| Echo lab uppercase extension | **Actual ASCII arithmetic error:** suggests adding 20H to lowercase for uppercase. | ASCII uppercase is obtained by *subtracting* 20H for ‘a’–‘z’; adding goes in the opposite direction for ‘A’–‘Z’. Corrected echo code bounds the range. |
| Echo timing extension | **Simplified latency model:** implies every character must be 2 ms apart. | Isolated round-trip wire time is roughly two frame times; full-duplex receive/transmit can overlap. Throughput is limited by the actual schedule/buffering, not by a universal doubled frame period. |

| HTML location | Finding / classification | Resolution and evidence |
|---|---|---|
| Initialisation snippets | **Teaching simplifications:** omit headers/helpers, initialise TH1 only, replace all TMOD, depend on reset SMOD. | Complete examples provide headers/helpers, load TH1/TL1, preserve Timer 0 and explicitly set SMOD. A TH1-only setup has a longer first interval but correct subsequent reload rate. |
| Voltage/cable diagrams | **Simplified convention:** 0/5 V logic and a fixed crossing rule. | Real VOH/current and converter requirements must be checked; label the PC DTE pin convention and cable path. The HTML's logic-level plot mixes nominal output levels and thresholds. [S51], [MAX]. |
| Scope/coverage | **Coverage limitation, not hardware error:** HTML develops serial/interrupts but not full Module 3. | Official physical syllabus p. 129 adds LCD, ADC/DAC, relays, stepper, DC motors and PWM. These are supplied here with device-specific assumptions and references. |

**Verification limits.** No native C51 compiler or physical 8051 board was available. Device-specific analog behaviour, switch bounce, motor load/gear calibration and generated-code ISR timing require bench/native-toolchain work. Public textbook access was limited to available previews/contents; no unavailable chapter body or invented textbook page was treated as consulted evidence. Detailed source and validation records accompany the editable bundle.

# 11 Resources mapped to the syllabus

## 11.1 Prescribed textbooks: what was accessible

The course bibliography on physical syllabus page 131 lists the Mazidi text, Lyla B. Das's *Embedded Systems: An Integrated Approach*, Kenneth J. Ayala's *The 8051 Microcontroller*, Craig Steiner's 8051/8052 architecture text and Steve Furber's ARM system-on-chip text. It also lists Patel, Raj Kamal and Uma Rao/Pallavi as references. ARM material belongs primarily to the following module. Use the classic 8051 chapters for this guide.

**Access boundary.** The publicly accessible publisher preview of the Mazidi second-edition text contains front matter, detailed contents and introductory material. The map below was checked against its printed contents pages vi–vii; the Module 3 chapter bodies were not available there. The page numbers below are *reading destinations printed in that preview*, not claims that those chapter pages were read. Hardware details in this guide were independently checked using the primary manuals and datasheets below. No unavailable Ayala/Das/Furber chapter or page reference is invented.

### [MAZIDI] Verified second-edition chapter map

Muhammad Ali Mazidi, Janice Gillispie Mazidi and Rolin D. McKinlay, *The 8051 Microcontroller and Embedded Systems: Using Assembly and C*, second edition. Publisher catalogue; publisher preview with detailed contents.

| Guide topic | Textbook chapter/section | Printed start |
|---|---|---|
| C51 prerequisites | Ch. 7, Programming in C; 7.1 types/time delay, 7.2 I/O, 7.3 logic operations, 7.5 code ROM | 181; 182, 190, 194, 204 |
| Timer prerequisite | Ch. 9; 9.1 timers, 9.3 C timer programming | 239; 240, 260 |
| Serial / RS232 / C | Ch. 10; 10.1 basics, 10.2 RS232 connections, 10.5 serial programming in C | 277; 278, 285, 306 |
| Interrupts | Ch. 11; 11.1 architecture, 11.3 external, 11.4 serial, 11.5 priority, 11.6 C | 317; 318, 326, 333, 337, 340 |
| LCD | Ch. 12, 12.1 LCD interfacing. Keypad section is beyond the stated Module 3 list. | 351; 352 |
| ADC / DAC | Ch. 13; 13.1 ADC, 13.2 DAC. Sensor material is supplementary. | 373; 374, 398 |
| Relay / stepper / DC / PWM | Ch. 17; 17.1 relays/optoisolators, 17.2 stepper, 17.3 DC motor/PWM | 491; 492, 498, 507 |

**Craig Steiner.** The author's detailed contents verifies Ch. 8 timers, Ch. 9 serial communication (9.1–9.5 configuration/read/write), Ch. 10 interrupts (10.3 polling sequence, 10.4 priority, 10.7.2 external, 10.7.3 serial, 10.8 register protection), and 14.2.6/14.2.13/14.2.14 for the board's LCD/programming/ports. The publisher preview was accessible in the browser; it is not a substitute for access to all those chapters. This book emphasises architecture/assembly; translate concepts using official C51 rules.

For Ayala, use the edition's contents/index entries for serial ports, interrupts and interfacing. For Das/Raj Kamal, use embedded I/O, interrupt/event handling and peripheral chapters. Chapter numbering was not independently available for their prescribed editions, so this guide gives no guessed map.

## 11.2 Primary architecture and compiler documentation

Links were inspected during preparation on 7 October 2026. Manufacturer documentation defines device behaviour; classroom prose and this guide explain it. For a different part number or core, recheck its own documentation.

### [INTEL] Classic architecture and hardware behaviour

Intel MCS-51 Microcontroller Family User's Manual, university-hosted manufacturer manual. Use the classic hardware chapter: serial control (printed 3-15), baud generation (3-16), mode-1 reception (3-20), interrupt priority (3-24), and interrupt response/idle-power sections. This is the primary check for TI/RI timing, receive acceptance, flags and interrupt operation. Its physical PDF pages differ from printed chapter page labels.

### [NXP] Classic 80C51 family hardware description

NXP official archive page and attached hardware description. Compare the serial-port modes, interrupt enable/priority registers, response and I/O-port sections. CMOS power-control features and 8052 Timer 2 additions are labelled separately.

### [S51] Selected classic-compatible hardware example

Microchip/Atmel AT89S51 datasheet. Pin descriptions, SFRs, interrupt structure, port characteristics and DC electrical limits support the GPIO examples. Its watchdog, dual data pointers and POF are derivative features; they do not redefine the original 8051.

### [KEILIRQ] C51 interrupt functions and vector numbers

Keil Application Note 103: Interrupt Handling in C51. Check `interrupt n`, `using n`, vector numbering and compiler-generated context handling. The hardware vector address and C51 function number are distinct representations of the same entry.

### [KEILTYPE] Type qualifiers and scalar sizes

Keil C51 type modifiers: volatile/const; Keil integer-type size/range table. Volatile prevents optimisation from assuming a shared object is unchanged; it does not supply atomicity or queue capacity.

### [KEILMEM] Memory placement and pointer conventions

Keil Application Note 101: Memory Space Utilization in C51; Keil C51 pointer storage sizes. Consult for `data`, `idata`, `xdata`, `code`, memory models and generic versus memory-specific pointers. These are compiler conventions, not standard ISO C keywords.

**[KEILBANK] Register-bank selection**

Keil C51 register-bank specification. Check the compiler's rules before using `using`; a bank shared by nested handlers can corrupt saved computation.

**[KEILREENT] Reentrant functions**

Keil C51 reentrant functions. Explains simulated-stack handling and the cost of reentrancy. The examples keep ISR calls small and avoid sharing ordinary helper routines across preempting contexts.

## 11.3 Device references for each interface

**[MAX] Logic UART to RS232**

TI MAX232 datasheet: pin functions, receiver thresholds, driver levels, charge-pump capacitors and application circuit. The selected MAX232 uses 1 $\mu$F capacitors; another variant's values must not be substituted automatically.

**[LCD] Character LCD controller**

Hitachi HD44780U manufacturer datasheet, Adafruit-hosted copy: instructions/DDRAM, write/read timing and instruction-based initialisation. The controller datasheet does not determine an arbitrary module's backlight resistor or connector arrangement.

**[ADC] Parallel SAR ADC**

TI ADC0801–ADC0805 / ADC0804-N datasheet: pin mapping, start/read waveforms, INTR-reset timing note, RC clock, reference scaling and transfer curve. This guide selects the ADC0804 variant.

**[DAC] Current-output DAC**

TI DAC0808 datasheet: A1–A8 significance, reference-current network, compensation capacitor and output compliance/current direction.

**[HCT244] Non-inverting parallel buffer**

TI SN74HCT244 datasheet: TTL input thresholds, pin grouping, output drive and active-low enables. It buffers the DAC bus; the firmware's byte significance is preserved.

**[OPAMP] Selected DAC I/V amplifier**

TI TL081 family datasheet: supply, input common-mode and output-swing requirements. The example uses dual $\pm 12$ V supplies and a grounded summing-node reference.

**[HCT14] Reset-safe inverting driver buffer**

TI SN74HCT14 datasheet: TTL-compatible Schmitt inputs and output drive. Reset-high MCU inputs produce low load-control outputs. Set unused inputs to defined levels; supply pin 14=+5 V, pin 7=ground.

**[RELAY] A concrete 5 V coil**

Omron G5V-1 relay datasheet: DC5 coil resistance/current, operating voltage and contact ratings. Coil and contact ratings describe different circuits.

**[NPN] Discrete relay switch**

onsemi P2N2222A datasheet: collector/base/emitter pin order, current ratings and saturation-test conditions. A transistor with a similar name can have another package pin order.

**[DIODE] Flyback-diode family**

Vishay 1N4001–1N4007 datasheet: current and reverse-voltage ratings. A 1N4007 is the selected slow-relay flyback example; cathode to positive coil supply.

**[ULN] Stepper coil current driver**

TI ULN2003A datasheet: Darlington channel input/current limits and clamp COM pin. COM is connected to the positive coil supply, while the emitter ground pin joins logic ground. The preceding HCT buffer addresses the classic GPIO's weak high-source drive.

**[STEPPER] Selected geared unipolar motor**

Kiatronics/Welten 28BYJ-48 5 V datasheet, MikroElektronika-hosted copy. The nominal stride/gearing supports the labelled ideal 4096-half-step/2048-full-step model. Supplier gearbox ratios and wire order need calibration/identification.

**[BRIDGE] DC-motor H-bridge**

TI L293/L293D datasheet: enable truth table, motor and logic supply pins, current/thermal limits and protection. L293D includes clamp diodes and is rated differently from L293; the selected motor's stall current must satisfy the whole driver design.

## 11.4 Supplementary university study material

**[UGC] Interrupt programming in Embedded C**

UGC e-PG Pathshala / INFLIBNET: Interrupt Programming in Embedded C. Useful for step-by-step C examples and exam explanation; primary architecture/C51 sources settle ambiguous flag and compiler behaviour.

NPTEL: Microprocessors and Microcontrollers, IIT Kharagpur, Santanu Chattopadhyay. Use the course's 8051 I/O, timer, serial and interrupt lectures for another university explanation. The prescribed chapter map above is the direct route for this module; this course is broader.

**A disciplined final reading pass**

For each interface, redraw the selected connections, explain the handshake/current path, derive one numerical and trace its program. For serial/interrupt questions, check flag timing, source enable, global enable, vector, priority and ownership. Treat datasheet limits as requirements and illustrative timing/current assumptions as assumptions.